LUNZ RESEARCH GROUP
LRG / PROGRAMA DE PESQUISA E ENGENHARIA

LRG SOVEREIGN AGI

Adaptive Cognitive Intelligence Architecture (ACIA)
Plano governamental de pesquisa, desenvolvimento, engenharia e implantação

VERSÃO 1.0 | 03 DE NOVEMBRO DE 2026

Documento-base para avaliação técnica, científica, jurídica e orçamentária.
Horizonte proposto: 60 meses. Âmbito inicial: Brasil.

Uma arquitetura cognitiva artificial integrada, multimodelo, relacional, adaptativa e experimentalmente autoaperfeiçoável.

Aviso de natureza, maturidade e uso do documento

Este documento converte a visão de pesquisa do Lunz Research Group em um programa técnico verificável. Não constitui demonstração de AGI, especificação executiva de obra, projeto básico de contratação, parecer jurídico ou autorização para operar sistemas governamentais críticos.

Classificação de maturidade

- Praticável com tecnologia atual: integração multimodelo por API, execução local de modelos com pesos legalmente acessíveis, memória persistente, grafos de conhecimento, avaliação automatizada, fine-tuning controlado, LoRA, observabilidade, isolamento e trilhas de auditoria.
- Pesquisa em aberto: interpretação abrangente de circuitos neurais, descoberta robusta de representações causais, aprendizagem contínua com baixa interferência, generalização ampla de alterações paramétricas e autoaperfeiçoamento que aumente capacidade geral sem regressões.
- Não demonstrado: consciência, experiência subjetiva, inteligência geral artificial, melhoria recursiva ilimitada ou capacidade de inferir semântica inequívoca de parâmetros isolados.

Regras de uso

As estimativas financeiras são envelopes de planejamento, não cotações. Os nomes e versões de modelos refletem páginas oficiais consultadas em 03/11/2026 e devem ser novamente verificados antes da contratação. Leis e atos infralegais devem ser revisados pela assessoria jurídica do órgão em cada caso concreto.

> Princípio de projeto
>
> O sistema é uma plataforma de apoio à pesquisa e à decisão. Não substitui autoridade pública, responsabilidade profissional, deliberação democrática ou devido processo.

Elaboração técnico-documental: arquitetura de referência ACIA / LRG. Atribuições, custos e entregáveis devem ser ratificados por instituição contratante e responsáveis técnicos habilitados.

Sumário

Aviso de natureza, maturidade e uso do documento	2
Sumário	3
Resumo executivo	6
1. Identidade institucional e objetivos	7
2. Princípios de arquitetura e requisitos não funcionais	8
3. Arquitetura geral ACIA	9
4. Interfaces e contratos entre módulos	10
5. Orquestrador multimodelo	11
6. Consórcio de modelos de fronteira: panorama em 03/11/2026	12
6A. Contexto, modalidades e preços públicos de referência	13
7. Treinamento com professores múltiplos	14
8. Weight Intelligence Engine (WIE)	15
9. WIE: métricas de espaço de parâmetros	16
10. WIE: ativações, circuitos e causalidade experimental	17
11. Weight-to-Markdown (W2M)	18
12. Motor RAGI: representação relacional	19
13. RAGI: inferência, transitividade e contradição	20
14. RAGI: consulta, atualização e controles	21
15. Computational Affect Engine: estados computacionais	22
16. Affect Engine: dinâmica e política de controle	23
17. Cognitive Memory System	24
18. Reasoning Engine	25
19. Metacognition Engine e calibração	26
20. World Model e transições de estado	27
21. Simulation Engine	28
22. Self-Improving Cognitive System	29
23. Mecanismos de melhoria e recursividade	30

24. Aprendizagem contínua e proteção contra esquecimento 31

25. Estrutura de software e repositório 32

26. Stack tecnológica recomendada 33

27. Interfaces e API de referência 34

28. Exemplo Python: contrato de experimento 35

29. Manual de implementação: ambiente inicial 36

30. Manual de implementação: integração dos motores 37

31. Infraestrutura: dimensionamento por cenário 38

32. Infraestrutura: energia, refrigeração e continuidade 39

33. Plano financeiro de 60 meses: premissas 40

34. Orçamento comparativo: três cenários 41

35. Perfil de desembolso e custo total de propriedade 42

36. Equipe e estrutura institucional 43

37. Governança e órgãos de decisão 44

38. Segurança arquitetural e threat model 45

39. Segurança do ciclo de autoaperfeiçoamento 46

40. Conformidade jurídica no Brasil: mapa inicial 47

41. Conformidade internacional e contratação pública 48

42. Aplicações governamentais prioritárias e limites 49

43. Programa experimental: desenho comum 50

44. Experimentos A–D 51

45. Experimentos E–H 52

46. Fluxo de avaliação e indicadores-chave 53

47. Cronograma de 60 meses: mapa de fases 54

48. Gestão de riscos do programa 55

49. Arquitetura de implantação governamental 56

50. Plano de validação operacional do piloto 57

51. Resultados esperados e limites científicos 58

52. Recomendações executivas e decisão solicitada 59

Conclusão 60

Referências bibliográficas e fontes oficiais (1/3) 61

Referências bibliográficas e fontes oficiais (2/3) 62

Referências bibliográficas e fontes oficiais (3/3) 63

Apêndice A. Glossário de termos e maturidade 64

Apêndice B. Formalização consolidada 65

Apêndice C. Checklist de revisão técnica e pré-produção 66

Apêndice D. Questões de pesquisa que permanecem abertas 67

Resumo executivo

O LRG SOVEREIGN AGI propõe uma arquitetura cognitiva modular na qual modelos fundacionais são recursos especializados, e não a totalidade do sistema. A ACIA combina orquestração multimodelo, análise neural, relações explícitas, estados afetivos computacionais, memória, raciocínio, metacognição, modelos de mundo e um mecanismo de experimentação com mudanças reversíveis.

Tese de engenharia

A hipótese operacional é que um sistema composto por modelos heterogêneos, memória estruturada, verificadores externos e ciclos de melhoria controlados pode superar um modelo isolado em determinadas famílias de tarefas. Isso é hipótese mensurável, não uma conclusão sobre AGI.

Recomendação principal

- Começar por um laboratório modular com 8 a 16 aceleradores, 25 a 40 profissionais equivalentes em tempo integral e orçamento de prova de conceito.
- Construir primeiro o contrato de dados, o orquestrador, o banco de evidências, a avaliação e o sistema de rollback. Depois conectar motores cognitivos.
- Usar modelos proprietários por API como professores ou especialistas e modelos de pesos disponíveis sob licença para análises internas.
- Separar o ciclo de pesquisa do ambiente de produção. Toda melhoria passa por testes cegos, regressão, revisão independente e aprovação humana.

Resultados de 60 meses

O resultado contratável é um sistema experimental auditável, com módulos integrados, benchmarks internos, pilotos controlados e evidência quantificada. A meta não é declarar AGI por cronograma, mas avaliar se as capacidades observadas justificam ampliar a pesquisa.

> Decisão proposta
>
> Aprovar uma Fase 0 de seis meses, com governança, arquitetura executável mínima, mapa jurídico, inventário de dados, benchmark de referência e orçamento refinado. A expansão depende de critérios de saída definidos previamente.

1. Identidade institucional e objetivos

Nome do programa: LRG SOVEREIGN AGI. Nome técnico da arquitetura: Adaptive Cognitive Intelligence Architecture (ACIA). Instituição proponente conceitual: Lunz Research Group (LRG). Natureza: programa independente de pesquisa e engenharia, com implantação potencial em infraestrutura pública sob contrato e governança institucional.

Objetivo central

Investigar uma arquitetura capaz de integrar modelos de IA heterogêneos, analisar estruturas internas acessíveis, aprender de forma incremental, construir representações relacionais, modelar estados do ambiente, avaliar limitações próprias e testar aperfeiçoamentos por experimentos reproduzíveis.

Objetivos mensuráveis

- Demonstrar redução de erro ou custo em tarefas científicas e administrativas delimitadas, comparada a controles isolados.
- Manter proveniência de cada evidência, transformação, inferência e decisão de roteamento.
- Executar experimentos de interpretabilidade e alterações de modelo sob autorização explícita.
- Medir retenção de competências anteriores após atualizações contínuas.
- Calibrar confiança e abstenção, minimizando afirmações sem sustentação.
- Demonstrar que qualquer melhoria implantada pode ser revertida para uma versão íntegra conhecida.

Definição operacional de sucesso

Sucesso significa melhoria replicável em um conjunto diversificado de tarefas, com incerteza reportada, custo operacional aceitável, segurança testada, ausência de regressões acima do limite aprovado e supervisão humana efetiva. Não se define por fluência conversacional nem por uma pontuação única de benchmark.

2. Princípios de arquitetura e requisitos não funcionais

A ACIA será modular na implementação, mas integrada em seus fluxos de dados e validação. Cada motor poderá formular hipóteses e produzir resultados independentes; o núcleo de integração resolve conflitos usando evidências, verificações e regras explícitas.

Princípios

- Proveniência por padrão: nenhuma afirmação importante deve perder fonte, data, versão do modelo, contexto ou método de extração.
- Separação entre dado e instrução: conteúdo recuperado, documentos e saídas de modelos são dados não confiáveis; não podem alterar políticas do sistema.
- Falha segura: falta de autorização, inconsistência crítica ou baixa confiança leva à abstenção, revisão ou execução em sandbox.
- Reversibilidade: versões de modelos, prompts, adaptadores, bases, políticas e índices são imutáveis e endereçados por hash.
- Portabilidade: interfaces internas neutras reduzem dependência de fornecedor; nenhuma API externa é requisito único.
- Observabilidade: latência, custo, qualidade, erros, uso de dados e incidentes são registrados com política de retenção.

Requisitos não funcionais iniciais

SLOs de referência para o protótipo: disponibilidade mensal de 99,5% no plano de controle; 99% dos eventos internos com trace correlacionável; restauração do plano de controle em até 4 horas; perda máxima de 1 hora de metadados após incidente. Esses valores devem ser recalibrados por criticidade e não se aplicam automaticamente a inferência de provedores externos.

> Regra de fronteira
>
> A camada cognitiva nunca tem permissão direta para modificar IAM, políticas de rede, mecanismos de auditoria, chaves, limites de ferramenta ou configuração de segurança.

3. Arquitetura geral ACIA

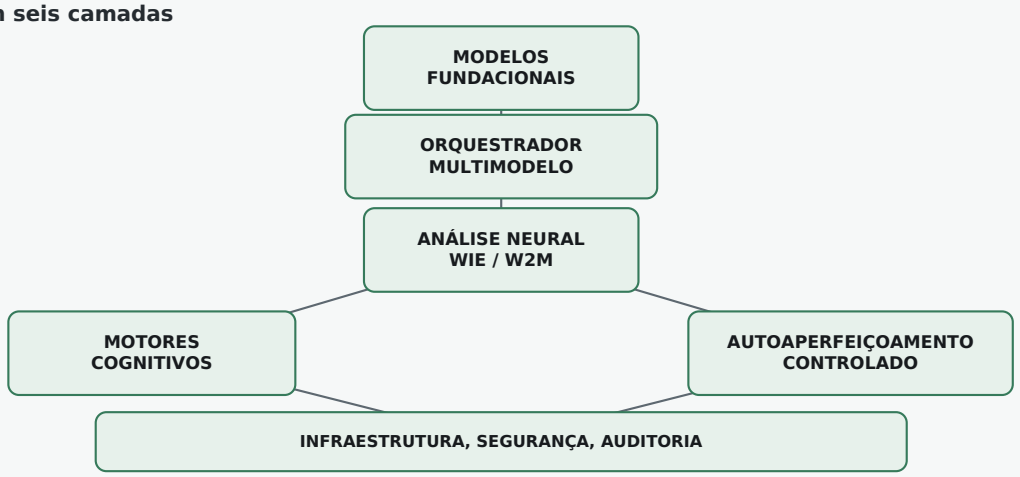
A arquitetura é organizada em seis camadas lógicas. As camadas não correspondem necessariamente a seis clusters ou seis processos: podem ser implementadas como serviços independentes, bibliotecas ou workers, dependendo da escala e do risco.

Camada 1 - Modelos fundacionais: APIs proprietárias, pesos abertos legalmente utilizáveis, modelos científicos, matemáticos, de código e multimodais. Camada 2 - Orquestrador: seleção, execução paralela, medição de divergência, fusão de evidências e custeio. Camada 3 - WIE: metadados, pesos, ativações e interpretabilidade. Camada 4 - motores cognitivos: relações, afetos computacionais, memória, raciocínio, planejamento, aprendizagem e metacognição. Camada 5 - melhoria: hipóteses, branches experimentais, validação, aprovação e rollback. Camada 6 - plano operacional: computação, armazenamento, segurança, governança e interfaces.

Dependências críticas

- Os motores cognitivos dependem de contratos de dados comuns, identidade persistente de entidades, relógio lógico e proveniência.
- O sistema de melhoria depende de avaliações congeladas, ambientes isolados, versionamento de artefatos e aprovadores independentes.
- O WIE depende de acesso autorizado aos artefatos do modelo e não pode inferir parâmetros internos de serviços por API.
- A camada de governança atravessa todas as demais e registra autorização, retenção, acesso, revisão e incidentes.

Visão em seis camadas



NOTA DE ENGENHARIA

O diagrama representa dependências lógicas. Em produção, plano de controle, dados e execução devem ser segregados por domínio de segurança.

4. Interfaces e contratos entre módulos

Para evitar um conjunto de agentes acoplados por texto livre, a ACIA deve usar envelopes de evento e objetos tipados. Texto natural continua sendo útil para interação, mas não deve ser o único formato de integração.

Envelope mínimo de mensagem

```
Event = {event_id, trace_id, timestamp, producer, schema_version, payload, provenance,
sensitivity, policy_tags}
```

Cada payload possui schema versionado. Campos desconhecidos são rejeitados ou encaminhados para compatibilidade explícita; não são reinterpretados silenciosamente.

Objetos de domínio

- Task: objetivo, critério de sucesso, limites, prazo, orçamento de tokens e nível de risco.
- Evidence: claim, fonte, trecho ou hash, data, método, confiança calibrada, dependências e validade.
- Hypothesis: enunciado falsificável, previsões, condições de teste, evidência favorável e contrária.
- Experiment: versão-base, branch, dataset, seed, métricas, ambiente, resultado e decisão.
- ModelArtifact: fornecedor, model_id, snapshot, licença, hash, arquitetura conhecida, endpoint ou caminho de pesos.

Protocolos

HTTP/gRPC para APIs de serviço; filas duráveis para tarefas longas; eventos com schema registry; SQL para transações; armazenamento objeto para artefatos; formatos de grafo para relações. As chamadas entre motores devem carregar trace_id e autorização delegada de escopo mínimo.

5. Orquestrador multimodelo

O orquestrador é um roteador de especialistas e avaliadores. Seu objetivo não é perguntar a todos os modelos sempre, mas selecionar o conjunto mínimo que maximize qualidade esperada sob restrições de custo, latência, disponibilidade e segurança.

Fluxo de execução

- Classificar tarefa, risco, modalidade, idioma, sensibilidade e necessidade de dados atualizados.
- Escolher modelo primário e verificadores com base em avaliações próprias e dados de produção.
- Executar em paralelo quando o ganho esperado superar custo e latência.
- Extrair claims atômicos, passos verificáveis, códigos, fontes e incerteza de cada saída.
- Executar checagens independentes: compilação, execução em sandbox, consulta de fontes, verificação formal, cálculo ou busca documental.
- Resolver divergências por evidência, especialização e teste; votação isolada não é critério suficiente.

Roteamento formal

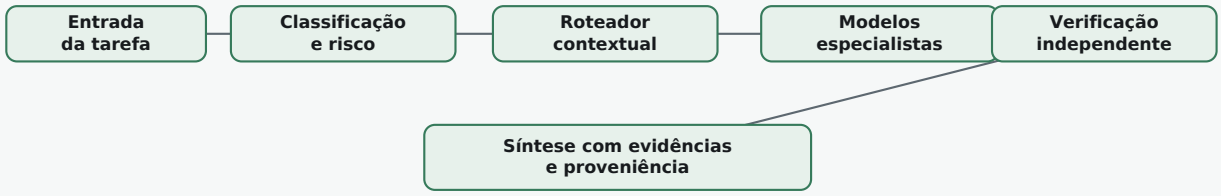
$$s_i(x) = \beta_1 Q_i + \beta_2 S_i + \beta_3 V_i - \beta_4 C_i - \beta_5 L_i - \beta_6 R_i$$

$$\alpha_i(x) = \exp(s_i/\tau) \,/\, \Sigma_j \exp(s_j/\tau)$$

Q é qualidade medida; S, adequação de especialidade; V, validade/disponibilidade; C, custo normalizado; L, latência; R, risco operacional. Os β são ajustados sobre dados de avaliação, com restrições por política. A distribuição α serve para escolher ou alocar consulta, não para afirmar que probabilidades token a token de modelos distintos podem ser somadas.

> Correção técnica
>
> A expressão $P(y|x) = \Sigma \alpha_i P_i(y|x)$ só tem sentido probabilístico direto se os modelos compartilham um espaço de saída e os eventos y são comparáveis. Para modelos heterogêneos com tokenizadores e APIs diferentes, consolidar evidências semânticas estruturadas.

Fluxo de decisão do orquestrador



O plano de controle aplica autorização, limites de custo e política antes de chamar qualquer ferramenta.

6. Consórcio de modelos de fronteira: panorama em 03/11/2026

O inventário abaixo é uma lista inicial para benchmark e não um ranking. Nomes, endpoints, contexto, modalidade, preços e licenças mudam com frequência. A seleção efetiva deve ser refeita por um job semanal de catálogo e por validação contratual, sem codificar nomes de versões no núcleo ACIA.

A tabela registra o que foi possível confirmar em páginas oficiais consultadas na data de corte. Em cada fornecedor, a arquitetura deve registrar modelo, identificador, snapshot, região, licença, subcontratantes, política de dados, disponibilidade e data da checagem.

Política de procurement

Manter pelo menos dois caminhos tecnológicos por função crítica: um endpoint externo e um modelo implantável sob controle local, quando tecnicamente possível. A substituição de fornecedor exige teste de equivalência, não mera troca de nome.

Laboratório	Modelos observados	Modo de acesso observado	Verificações obrigatórias
OpenAI	GPT-6 Astra; GPT-6.1 Sol; GPT-6.1 Terra; GPT-6 Luna	API proprietária; IDs oficiais [1]	Verificar termos de retenção, região, logging, limites e condições enterprise.
Anthropic	Claude Fable 5.1; Opus 5.5; Sonnet 5.5; Haiku 5.5; Mythos 5.1 (organizações verificadas)	API; Mythos 5.1 sujeito a programa de verificação [2]	Revisar uso de dados, versão, datas de aposentadoria e restrições contratuais.
Google DeepMind	Gemini 4 Argon; Gemini 3.8 Flash; Gemini 3.8 Audio Live	API e canais Google; model card Argon [3]	Não fixar alias preview em produção; monitorar depreciação.
Meta AI	Muse Wonder; Muse Wonder Full; Muse Spark 1.3; Llama 4	Acesso por produtos/serviços; disponibilidade de API e pesos deve ser verificada por modelo [4]	“Open weights” não equivale automaticamente a open source; revisar licença.
xAI	Grok 4.7	API oficial; modelo e preços divulgados [5]	Confirmar condições para dados governamentais e ferramentas de busca.
DeepSeek	V4.1 Flash-1017; rotas/aliases anteriores sujeitos a redirecionamento	API; alias deepseek-flash documentado [6]	A changelog reporta redirecionamento de chamadas V4 Pro para V4.1 Flash; validar ID efetivo e preço no momento da compra. Não presumir pesos disponíveis.
Alibaba / Qwen	Qwen3.8-Max; Qwen3.7-Plus; Qwen3.8-Flash	API; também há modelos da família com pesos [7]	Separar catálogo próprio de modelos terceiros hospedados no Model Studio.
Mistral	Mistral Large 4; Mistral Small 4	Large 4 em preview público, licença Open Weight; disponibilidade do checkpoint deve ser reconfirmada. Small 4 sob Apache 2.0 [8]	Verificar os termos da licença “Open Weight” e limites de redistribuição.
Moonshot AI	Kimi K3 multimodal	Pesos disponíveis sob licença Kimi K3 e acesso via API/parceiros [9]	Acesso a pesos não inferido de presença em marketplace.
Z.ai / Zhipu	GLM 5.3; GLM 5.3 Flash	API e modelos distribuídos em plataformas parceiras [10]	Confirmar repositório oficial, hash e licença por checkpoint.
MiniMax	M3.1 Flash	Plataforma/API e plano de uso [11]	Avaliar custos reais por requisição e retenção antes de dados sensíveis.
NVIDIA	Nemotron 3.5 Super VL / Lightning; Nemotron 3 Ultra	Pesos, dados e receitas publicados; licença varia por artefato [12]	Confirmar versão exata, licença, requisitos e dependência de GPU.

NOTA DE ENGENHARIA

Acesso por API não autoriza inspeção de pesos. Repositório público ou marketplace não prova, sozinho, titularidade, licença comercial, integridade ou ausência de restrições.

6A. Contexto, modalidades e preços públicos de referência

Esta matriz complementa o catálogo de fornecedores. Os valores são preços publicados para rotas específicas em 03/11/2026, em USD por 1 milhão de tokens de entrada/saída, antes de impostos, câmbio, chamadas de ferramentas, modalidades não textuais, cache especial, descontos e acordos enterprise. Não constituem cotação governamental.

Fornecedor / modelo	Contexto e entrada	USD / 1M tokens (entrada / saída)	Acesso e observações
OpenAI — GPT-6 Astra	1,05M; texto e imagem	$10 / $50	API; até 128K tokens de saída. GPT-6.1 Sol: $2/$10; Terra: $1/$5; Luna: $0,10/$0,50. [1]
Anthropic — Fable 5.1 / Opus 5.5 / Sonnet 5.5 / Haiku 5.5	1M; texto e imagem	$10/$50; $4/$20; $2/$10; desde $0,10/$0,50	API e nuvens parceiras; 128K de saída. Mythos 5.1 exige verificação. [2]
Google — Gemini 4 Argon	1M combinado; texto, imagem, áudio e vídeo	Preço não confirmado na model card consultada	API e Google AI Studio; recuperar a tabela regional de preço antes da contratação. [3]
xAI — Grok 4.7	500K; texto e imagem	$2 / $6	API. Para contexto ≥200K, a tabela publicada indica $4/$12. [5]
DeepSeek — V4.1 Flash-1017	1M; texto e imagem	$0,30/$1,20 pico; $0,15/$0,60 fora de pico	API; cache-hit $0,006/$0,003, respectivamente. Preço por janela horária UTC. [39]
Mistral — Large 4	1M; multimodal, incluindo visão	$1,36 / $4,18	API em preview público; licença Open Weight v26.10. Preço promocional expirado não foi usado. Verificar disponibilidade do checkpoint. [8]
Moonshot — Kimi K3	1M; texto e imagem	$3 / $15 (AWS Bedrock Global Standard)	Preço do canal Bedrock; API direta e outras regiões podem divergir. Pesos disponíveis sob licença própria Kimi K3. [40]
Qwen, Z.ai, MiniMax, NVIDIA e Meta	Varia por modelo: texto, visão, áudio e/ou vídeo	Preço público comparável não consolidado nesta matriz	Consultar portal e região por modelo; para pesos locais, modelar TCO de GPU, energia, storage, suporte e equipe. [4], [7], [10]–[12]

NOTA DE ENGENHARIA

A comparação de API deve usar o mesmo workload, janela, cache, região, modalidade, limites e qualidade exigida. O preço por token não é custo por tarefa concluída; a métrica de compra é custo total por tarefa aceita e auditável.

7. Treinamento com professores múltiplos

Modelos externos podem atuar como professores, avaliadores, red-teamers e geradores de hipóteses. O estudante ACIA, entretanto, deve ser treinado sobre dados selecionados e licenciados, com controle de proveniência, de duplicação e de qualidade.

Métodos candidatos

- Teacher–student distillation: transferir saídas ou distribuições quando o formato e a licença permitem; comparar desempenho antes/depois.
- Dataset generation: gerar itens sintéticos para cobrir lacunas, depois filtrar com verificadores e auditoria amostral humana.
- Preference optimization / RLHF: alinhar comportamento a preferências rotuladas, declarando a população e limitações dos rótulos [13].
- Reinforcement learning: usar recompensas verificáveis em ambientes delimitados; testar robustez contra reward hacking.
- Self-play e debate multiagente: explorar hipóteses e contraexemplos; não tratar consenso entre agentes correlacionados como verdade.
- Continual learning: replay, regularização, adaptadores, memória externa e testes de retenção; esquecimento catastrófico continua sendo problema aberto [14].
- Model merging: somente entre checkpoints compatíveis, arquitetura, parametrização e licença compatíveis; não fundir arbitrariamente arquiteturas diferentes.

Pipeline de dados

```
dados → licença → minimização → deduplicação → avaliação → geração/rotulagem → revisão → treino → teste cego → registro
```

Os dados sintéticos recebem rótulo de origem sintética. Um conjunto retido de teste é inacessível aos geradores e ao tuning para limitar contaminação. Dados de cidadãos não entram em treinamento por padrão; qualquer exceção requer base legal, avaliação de impacto, minimização, governança e contrato.

8. Weight Intelligence Engine (WIE)

O WIE é o laboratório de análise estrutural de modelos com pesos acessíveis. A unidade de trabalho não é “ler a mente do modelo”, mas testar hipóteses sobre parâmetros, ativações e circuitos em tarefas controladas.

Entradas e saídas

- Entradas: artefato do modelo, hash, arquitetura, tokenizer, licença, pesos, configuração, corpus de estímulos e ambiente de execução.
- Saídas: inventário de tensores, estatísticas por camada, normas, espectros, ativações, mapas de atenção com ressalvas, features de SAE, resultados de intervenções e relatórios Markdown.

Pipeline recomendado

1. Verificar proveniência e hash. 2. Validar formato e licença. 3. Carregar em sandbox sem rede. 4. Inventariar tensores e shapes. 5. Coletar estatísticas sem despejar pesos completos. 6. Selecionar estímulos de referência. 7. Capturar ativações por hooks. 8. Executar análises comparativas e causais. 9. Replicar em seeds/modelos. 10. Emitir relatório com hipóteses, evidências, incerteza e limitações.

Segurança do carregamento

Preferir formatos de tensores não executáveis e pesos com hash validado, por exemplo Safetensors, e cargas em modo restritivo. Nunca carregar checkpoints arbitrários com desserialização executável em ambiente com segredos ou acesso à rede. Imagens de contêiner e dependências também devem ser assinadas e verificadas.

> Limite estrutural
>
> Para modelo acessado apenas por API, o WIE só analisa comportamento observável, latência, erros, formatos e documentação oficial. Não deve tentar extrair pesos via engenharia reversa, exploração ou contorno de controles.

9. WIE: métricas de espaço de parâmetros

Para comparar versões é necessário alinhamento: mesma arquitetura, mapeamento equivalente de parâmetros, tokenizer compatível e política de serialização conhecida. Uma diferença de pesos entre famílias diferentes não é uma métrica direta de mudança cognitiva.

Distância global e por camada

$$D\theta = ||\theta_2 - \theta_1||_2 / (||\theta_1||_2 + \varepsilon)$$

A expressão acima é uma distância relativa. Para tensores de uma camada l, usar norma de Frobenius: D_l = ||W_l^(2) − W_l^(1)||_F / (||W_l^(1)||_F + ε). Registrar também o número de parâmetros alterados, sparsity da diferença e percentis de magnitude.

Similaridade de direção

$$\cos(\theta_1,\theta_2) = (\theta_1 \cdot \theta_2) / (||\theta_1||_2 \, ||\theta_2||_2)$$

Análise espectral

$$W = U\Sigma V^{T};\ E_k = (\Sigma_{i=1}^{k} \sigma_i^2) / (\Sigma_i \sigma_i^2)$$

A decomposição SVD permite estudar espectro singular, posto efetivo e concentração de energia em subespaços. A energia cumulativa E_k é uma característica descritiva, não uma explicação do que a camada “pensa”. Em matrizes grandes, usar aproximações iterativas com semente fixa e erro reportado.

Interpretação

Mudança paramétrica elevada não implica mudança comportamental proporcional; mudança pequena pode ter efeito grande em fronteiras sensíveis. Por isso, Dθ deve ser correlacionada com avaliação funcional e ativação, nunca utilizada como proxy isolada de capacidade.

10. WIE: ativações, circuitos e causalidade experimental

O estudo de ativações é mais informativo quando parte de tarefas e contrastes controlados. A análise mecanística contemporânea procura features, subespaços e circuitos em vez de atribuir significado simples a um neurônio isolado [15, 16].

Métricas funcionais

$$CKA(H_1,H_2) = ||H_1^T H_2||^2_F / (||H_1^T H_1||_F \, ||H_2^T H_2||_F)$$

CKA mede similaridade de representações sob determinadas condições. Também se devem medir distância de logits KL apenas quando o vocabulário e o alinhamento de saídas são comparáveis; taxa de acerto, calibração, consistência, robustez e custo são métricas funcionais independentes.

Activation patching / causal tracing

Definir um estímulo-base e um estímulo de contraste; registrar ativações de referência; intervir em uma posição/camada e medir alteração no logit ou na métrica-alvo. O efeito deve ser comparado com intervenções de controle, múltiplas posições e seeds. Uma ativação correlacionada não é automaticamente causa.

Sparse autoencoders

Treinar dicionários esparsos sobre ativações para decompor padrões distribuídos em features esparsas. Avaliar reconstrução, esparsidade, estabilidade entre seeds, interpretabilidade humana e causalidade das features. Evitar transformar descrições automáticas em rótulos factuais sem auditoria.

Relação entre mudança e desempenho

$$\rho = corr(\Delta \text{métrica_funcional}, \Delta \text{estatística_interna})$$

A correlação é exploratória. Para inferência causal, usar intervenção planejada, grupo de controle e ablação. Relatar intervalos de confiança, multiplicidade de testes e resultados negativos.

11. Weight-to-Markdown (W2M)

Weight-to-Markdown converte resultados analíticos em documentação legível e reprodutível. Não converte diretamente cada escalar de um tensor em prosa e não pretende atribuir semântica universal a pesos individuais.

Modelo de relatório

```
report.md = metadata + tensor_inventory + descriptive_stats + spectral_analysis +
activation_analysis + interventions + hypotheses + limits
```

Esquema do relatório

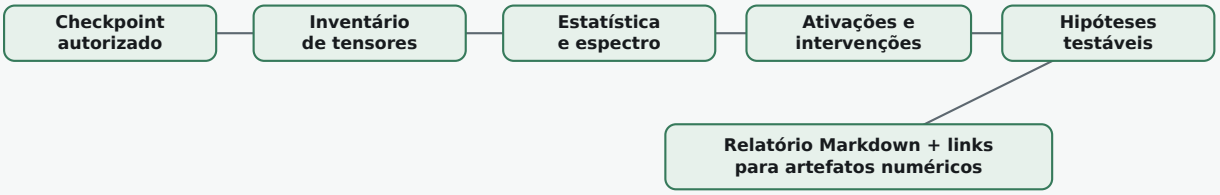
- Identificação: fornecedor, modelo, hash, checkpoint, tokenizer, versão da biblioteca, data e licença.
- Camada/parâmetro: nome, tipo, shape, dtype, quantidade de valores, device e política de acesso.
- Estatística: média, desvio, quantis, mínimo, máximo, norma L1/L2/Frobenius, sparsity e NaN/Inf.
- Análise: espectro aproximado, distribuição, ativação por estímulo, feature de SAE e resultado de patching.
- Hipótese: frase falsificável, evidência favorável/contrária, confiança calibrada e próximo experimento.
- Limitações: amostragem, não identificabilidade, confusão por tokenizer, alinhamento e ausência de causalidade.

Exemplo ilustrativo

Modelo: checkpoint de pesquisa A. Camada: bloco Transformer 12. Tensor: projeção de atenção. Shape: 4096 × 4096. Análise: quantis, norma de Frobenius, espectro singular aproximado e diferença contra snapshot anterior. Hipótese: mudança de subespaço associada à tarefa T. Evidência: efeito replicado em 3 seeds e 2 avaliações. Limite: associação experimental ainda não estabelece interpretação semântica geral.

> Regra de rastreabilidade
>
> Cada relatório deve apontar para dados e código que o produziram; o Markdown é um índice interpretável, não substituto do artefato numérico original.

Pipeline Weight Intelligence → documentação



Os pesos originais permanecem em artefatos numéricos imutáveis; o relatório resume evidências e limites.

12. Motor RAGI: representação relacional

O RAGI (Relational Artificial General Intelligence) é uma linha de pesquisa do LRG sobre relações como estrutura explícita de representação e descoberta. A hipótese é compatível com grafos de conhecimento e sistemas neuro-simbólicos, mas não prova que relações sejam o fundamento universal da inteligência.

Definição do grafo

```
G = (V, E, T, P)
```

V representa entidades e estados; E, arestas relacionais; T, tipos e assinaturas de relações; P, propriedades, contexto, evidência e proveniência. Cada aresta é um objeto versionado e pode possuir validade temporal, polaridade e status de revisão.

Esquema mínimo da relação

```
r = (id, source, target, type, evidence[], source_ref, confidence, context, valid_time,
status)
```

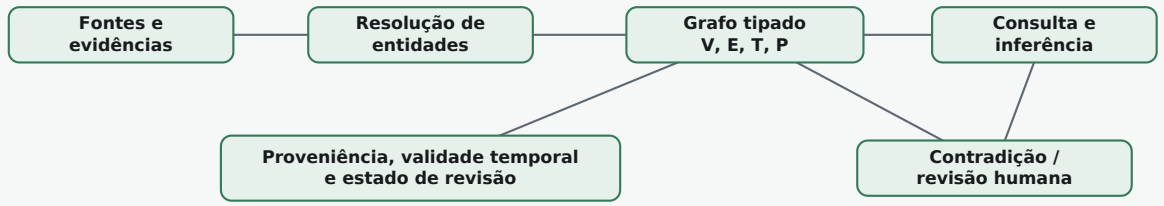
Identidade e incerteza

Entidades devem possuir identificadores internos e aliases. Desambiguação precisa de contexto e fonte. Confiança é uma estimativa calibrada do suporte à relação, não probabilidade universal de verdade. Evidências derivadas da mesma fonte não são independentes e não podem ser multiplicadas como se fossem observações distintas.

Relações e sistemas relacionados

O RAGI pode integrar knowledge graphs, ontologias, lógica de primeira ordem, redes Bayesianas, modelos causais e recuperação aumentada. A inovação proposta está na camada de descoberta, nos metadados de proveniência e na coordenação dos motores, não em alegar substituir essas disciplinas.

Fluxo de construção e validação do grafo RAGI



Relações derivadas são marcadas como inferidas e não substituem as evidências de origem.

13. RAGI: inferência, transitividade e contradição

Inferência relacional exige regras tipadas. A composição R(a,b) ∧ R(b,c) ⇒ R′(a,c) só é válida quando a semântica da relação e as restrições de domínio autorizam a composição.

Composição tipada

```
R₁(x,y) ∧ R₂(y,z) ∧ Compatible(R₁,R₂) ⇒ R₃(x,z)
```

Exemplo válido: se “é ancestral de” é transitiva e ambas as arestas foram verificadas, ancestral(A,B) ∧ ancestral(B,C) implica ancestral(A,C). Exemplo inválido: “reside em” não pode ser composto livremente para deduzir que A reside no país de B sem conhecer a relação município-país e sua semântica.

Confiança com dependência explícita

```
logit(p_new) = logit(p_prior) + Σ_j w_j · evidence_j
```

Os pesos w_j devem descontar duplicação, dependência de fonte, idade e qualidade. Caso a independência não seja plausível, usar modelo hierárquico ou agregador conservador e registrar o grafo de dependência da evidência.

Contradição

- Conflito lógico: predicados incompatíveis no mesmo contexto e intervalo.
- Conflito temporal: afirmações verdadeiras em períodos diferentes.
- Conflito de entidade: aliases ou entidades fundidas indevidamente.
- Conflito epistêmico: fontes com metodologias ou níveis de autoridade distintos.

Causalidade

Relações causais exigem hipóteses de identificação, ordem temporal, controle de confundidores, DAG ou intervenção. Coocorrência e correlação não bastam. O motor deve permitir status “não identificado” e manter hipóteses alternativas.

14. RAGI: consulta, atualização e controles

A camada RAGI deve ser usada por qualquer motor cognitivo, mas o resultado não deve ser tratado como verdade por simples presença no grafo. Cada consulta retorna afirmações, evidências, contraposições e validade.

Tipos de consulta

- Consulta de vizinhança: relações diretas, tipo, validade e proveniência.
- Consulta de caminho: caminhos compatíveis, limite de comprimento e score de suporte.
- Consulta de hipótese: relações esperadas que ainda não possuem evidência.
- Consulta de conflito: relações incompatíveis e conjuntos mínimos de evidência contraditória.
- Consulta temporal: fatos válidos em um instante ou intervalo.

Política de escrita

Somente serviços autorizados gravam relações. O motor de linguagem pode propor uma relação em estado “candidate”; o serviço RAGI valida tipo, fonte, duplicação, consistência e regra. Uma segunda aprovação é obrigatória para relações de causalidade, fatos de alto impacto ou integração de dados pessoais.

Métricas RAGI

```
Precision = TP/(TP+FP); Recall = TP/(TP+FN); ContradictionRate = conflitos / relações ativas
```

Medir também calibração Brier, cobertura de proveniência, tempo de atualização, taxa de relações expiradas que continuam ativas, qualidade de entidade e custo por inferência. Conjuntos de avaliação devem incluir relações falsas plausíveis e relações temporais.

> Critério de aceite
>
> O módulo somente avança quando suas inferências melhoram a recuperação ou o raciocínio em tarefas retidas sem aumentar falsos positivos além de um teto definido por domínio.

15. Computational Affect Engine: estados computacionais

O motor afetivo representa estados operacionais que modulam exploração, persistência e revisão. A terminologia de “emoção” é uma metáfora funcional: variáveis computadas não demonstram experiência subjetiva, consciência ou sentimento.

Vetor inicial

```
E_t = [c_t, u_t, q_t, f_t, s_t] ∈ [0,1]^5
```

c: propensão à exploração; u: incerteza estimada; q: urgência de tarefa; f: dificuldade/fricção observada; s: progresso/satisfação instrumental em relação a objetivo mensurável. A semântica de cada variável deve ser definida no contrato do motor e sujeita a auditoria.

Observáveis aceitos

- Incerteza: entropia calibrada, desacordo entre modelos, ausência de evidência ou intervalo preditivo.
- Urgência: prazo declarado, severidade de serviço ou regra de fila autorizada.
- Dificuldade: tentativas falhas, violações de invariantes, bloqueios e custo de busca.
- Progresso: distância a critério objetivo de conclusão.
- Curiosidade: valor esperado de informação ou ganho previsto de exploração.

Não permitido

O motor não deve perfilar emoções de cidadãos, otimizar dependência psicológica, manipular usuários ou usar sinais afetivos pessoais para decisão governamental. Seu escopo é o estado de trabalho da própria plataforma e a alocação de recursos.

16. Affect Engine: dinâmica e política de controle

Os estados precisam ser limitados, estáveis e explicáveis. A atualização deve depender de observações operacionais, objetivo ativo, limites e decaimento; não de interpretação antropomórfica de texto livre.

Modelo de atualização

$$E_{t+1} = \mathrm{clip}(\Lambda E_t + (I-\Lambda)g(O_t, G_t) - d \odot E_t,\ 0,\ 1)$$

Λ é diagonal com coeficientes de persistência em [0,1]; g produz um alvo calculado a partir de observações O e objetivo G; d é vetor de decaimento; clip limita componentes ao intervalo [0,1]. A estabilidade requer limites e atualização monotônica quando esperada.

Exemplo ilustrativo

Com $u_t=0{,}70$, alvo de incerteza 0,30, $\lambda=0{,}8$ e decaimento $d=0{,}02$: $u_{t+1} \approx 0{,}8\times0{,}70+0{,}2\times0{,}30-0{,}02\times0{,}70=0{,}546$. A variável diminui porque o sistema obteve evidência adicional; se a verificação falhar, o alvo pode aumentar.

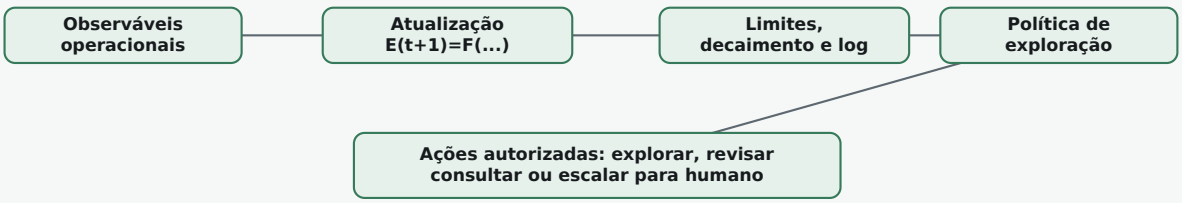
Política de ação

- Curiosidade alta + custo baixo: executar experimento exploratório pequeno.
- Incerteza alta + tarefa crítica: solicitar segundo verificador ou intervenção humana.
- Fricção crescente: parar de repetir a mesma estratégia e reformular o plano.
- Urgência alta: priorizar fila sem reduzir padrões de validação e segurança.

Indicador de qualidade

A política será considerada útil quando reduzir retrabalho e erros ou melhorar qualidade de decisão, sem aumentar custo desproporcional ou contornar validações.

Circuito operacional do motor afetivo



O vetor afetivo é estado de controle mensurável, não atribuição de sentimento subjetivo.

17. Cognitive Memory System

A memória é uma camada persistente que mantém contexto, resultados e conhecimento. Deve ser pesquisável, temporal, versionada e corrigível, com retenção proporcional à finalidade.

Subsistemas

- Memória de trabalho: estado de tarefa e artefatos temporários; TTL curto.
- Memória episódica: eventos, sessões, decisões e resultados experimentais.
- Memória semântica: conceitos e fatos com fontes e validade.
- Memória procedural: procedimentos aprovados, ferramentas e rotinas.
- Memória de pesquisa: hipóteses, experimentos, versões e resultados negativos.
- Memória relacional: entidades e arestas RAGI com proveniência.

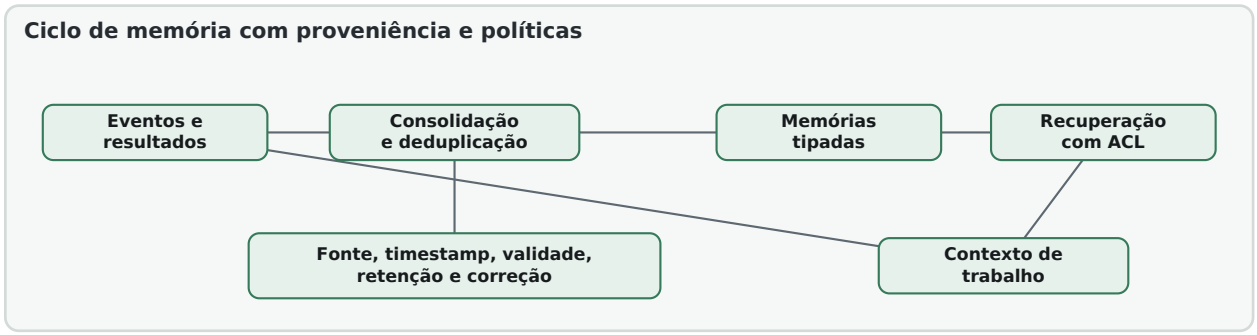
Consolidação

```
M_{t+1} = Consolidate(M_t, events, utility, confidence, retention_policy)
```

Consolidar não significa gravar tudo. Um processo agenda revisão, deduplica, verifica a fonte, identifica validade, anonimiza quando necessário e determina retenção. “Esquecimento” deve respeitar política e retenção legal, não apagar trilha de auditoria obrigatória.

Recuperação

Combinar pesquisa lexical, vetorial, temporal, relacional e filtros de acesso. Cada item recuperado deve retornar origem, data, score, política de acesso e trecho que sustenta a resposta. Memórias conflitantes devem ser apresentadas como conflito, não reconciliadas por texto plausível.



Cada resultado recuperado carrega proveniência, permissão e contexto temporal.

18. Reasoning Engine

O motor de raciocínio seleciona métodos de acordo com o tipo de problema. Modelos de linguagem podem propor passos; ferramentas matemáticas, executores e verificadores precisam validar quando o domínio permitir.

Métodos

- Dedução: aplicação de regras formais; boa auditabilidade quando premissas e lógica são explícitas.
- Indução: generalização a partir de amostras; deve reportar incerteza, cobertura e risco de distribuição diferente.
- Abdução: geração de explicações possíveis; requer comparação com alternativas e evidência discriminante.
- Probabilístico: posterior sobre hipóteses; depende da especificação do modelo e da qualidade dos priors.
- Matemático: álgebra simbólica, cálculo, provas assistidas e verificadores formais.
- Busca/planejamento: árvore, heurística, MCTS, SAT/SMT, programação matemática ou otimização.
- Simulação: avaliar consequências condicionais a um modelo; validade depende de calibração e hipóteses.

Seleção de método

```
m* = argmax_m [ expected_value(m|task) − λ·cost(m) − μ·risk(m) ]
```

O motor pode usar métodos em composição. Um problema de logística, por exemplo, pode combinar extração documental, modelagem de restrições, solver e explicação em linguagem natural. A resposta final deve distinguir resultado calculado, hipótese, simulação e parecer humano.

19. Metacognition Engine e calibração

Metacognição é implementada como controle explícito sobre qualidade do conhecimento, fontes, incerteza e decisão de continuar, consultar, abster-se ou pedir revisão. Não é prova de autoconsciência.

Registro metacognitivo

```
K = {claim, evidence, source_quality, freshness, uncertainty, contradiction, testability, decision}
```

Calibração

Medir Expected Calibration Error por bins, Brier score e cobertura seletiva. O sistema pode emitir “não sei” quando a cobertura de evidência é insuficiente. A calibração deve ser por tarefa, domínio, fornecedor, idioma e versão; um único score global esconde diferenças importantes.

$$\text{Brier} = (1/N)\ \Sigma_i\ (p_i - y_i)^2$$

Regras de decisão

- Consultar fonte atual quando a informação depende de data, preço, lei, modelo ou status de serviço.
- Usar verificador independente em conclusão matemática, código executável ou cálculo orçamentário.
- Abster-se se faltar autorização, fonte, cobertura ou estabilidade estatística.
- Escalar a humano quando risco, ambiguidade ou impacto sobre direitos ultrapassar limiar aprovado.

Avaliação

A avaliação deve testar se a confiança prediz acerto, se a abstenção reduz dano e se o motor reconhece erros induzidos. Não recompensar confiança verbal; premiar a calibração real em conjuntos retidos.

20. World Model e transições de estado

O modelo de mundo representa variáveis relevantes de um domínio, transições, tempo, dependências e incertezas. A ACIA não deve manter um “modelo universal” sem validação: modelos distintos serão treinados e testados para domínios específicos.

Formalização

$$s_{t+1} = T(s_t, a_t, \varepsilon_t);\ \varepsilon_t \sim P(\varepsilon \mid \text{contexto})$$

s é estado; a, ação ou intervenção; T, modelo de transição; ε, incerteza. As variáveis do estado devem ser observáveis ou acompanhadas de inferência. Para cenários governamentais, o modelo deve representar restrições institucionais, distribuição, sazonalidade e grupos afetados quando legalmente permitido.

Contrafactuais

Comparar s_{t+1} sob intervenções alternativas exige hipóteses explícitas e validação externa. Um contrafactual é “sob modelo M e premissas A, estimamos ...”, não um relato de fato. Modelos causais devem ser identificados por desenho de pesquisa e documentação.

Linha técnica

A literatura de world models explora representações comprimidas espaço-temporais e políticas aprendidas em ambientes simulados [17]. Para ACIA, adaptar os conceitos a sistemas de energia, mobilidade, ambiente e logística com dados rastreáveis e avaliação fora da amostra.

21. Simulation Engine

A engine de simulação converte hipóteses em experimentos executáveis. Ela deve acompanhar entradas, parâmetros, seed, versão, domínio de validade, incerteza e resultado.

Famílias de simuladores

- Matemáticos: álgebra, otimização, sistemas dinâmicos e equações diferenciais.
- Físicos/engenharia: modelos de energia, materiais, fluxo, manutenção e confiabilidade.
- Computacionais: filas, redes, escalonamento, desempenho e falhas.
- Agentes e cenários: comportamento agregado em ambiente sintético, com limites documentados.
- Políticas públicas: microssimulação, avaliação de impacto, sensibilidade e cenários.

Protocolo de execução

```
SimulationRun = (model_id, data_version, parameters, seed, horizon, assumptions, outputs, uncertainty)
```

Rodar baseline, cenários alternativos e análises de sensibilidade; comparar com dados históricos; verificar extrapolação; e publicar intervalos, não apenas estimativas pontuais. Não usar simulações como fatos observados nem como justificativa única para decisões que afetem direitos.

Métricas

Erro de previsão, cobertura de intervalos, estabilidade a perturbações, sensibilidade a parâmetros, diferença contra baseline e adequação fora do domínio. O relatório inclui observações que falsificariam o modelo.

22. Self-Improving Cognitive System

O ciclo de autoaperfeiçoamento é um processo de engenharia experimental, não uma permissão para que o sistema altere livremente a si mesmo. A versão de produção permanece imutável durante a experimentação.

Ciclo operacional

1. Medir resultado e erro. 2. Localizar gargalo por evidência. 3. Consultar memória e RAGI. 4. Formular hipótese falsificável. 5. Gerar branch de software, prompt, regra, memória ou adaptador. 6. Executar teste unitário, regressão e avaliação independente. 7. Comparar com baseline. 8. Aprovar por comitê técnico e autoridade designada. 9. Implantar canário com monitoramento. 10. Reverter automaticamente ou manualmente se limites forem violados.

Função objetivo

```
J(θ)=w_A A+w_G G+w_R R+w_C C−w_K K−w_D D
```

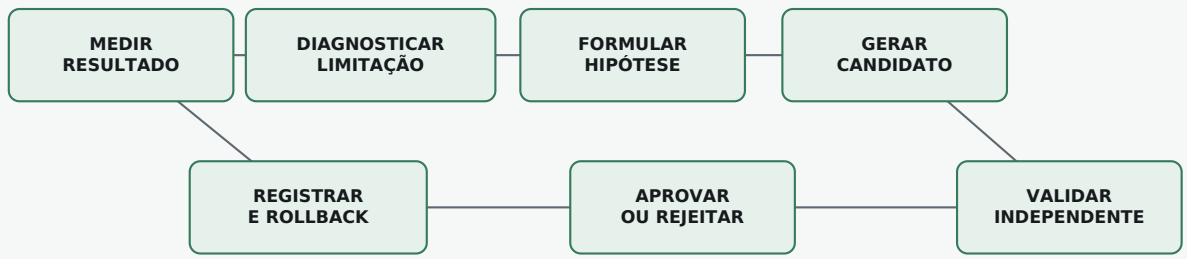
A: acurácia/qualidade; G: generalização retida; R: confiabilidade/calibração; C: consistência; K: custo/latência normalizados; D: degradação de capacidades anteriores. Todos os termos são normalizados e medidos por conjuntos específicos. Pesos w são definidos antes dos experimentos e submetidos a análise de sensibilidade.

Restrição

```
maximize J(θ) subject to Safety(θ) ≥ τ_s, Cost(θ) ≤ B, Regression(θ) ≤ δ
```

Se não houver ganho estatisticamente significativo, a modificação não é aprovada. Se aumentar uma capacidade e degradar uma anterior além de δ, a hipótese é rejeitada ou reformulada.

Ciclo de melhoria controlada



23. Mecanismos de melhoria e recursividade

A ACIA deve testar se melhorias em componentes cognitivos podem ser acumuladas, sem pressupor que a recursividade leve automaticamente a AGI ou a crescimento exponencial de capacidade.

Objetos passíveis de melhoria

- Estratégias de decomposição e verificação de raciocínio.
- Algoritmos de recuperação, memória e deduplicação.
- Políticas de roteamento e seleção de verificadores.
- Planejadores, simuladores e heurísticas.
- Prompts de sistema, templates, ferramentas e adaptadores treináveis.
- Pesos de modelos sob autorização, com teste robusto e retenção.

Métricas de generalidade

Melhoria geral exige benefício em múltiplas famílias de tarefas não vistas, transferência para novos domínios, robustez a mudanças de distribuição e baixo custo de supervisão. Uma melhoria local em código ou benchmark não é melhoria geral de inteligência.

Impedimentos previsíveis

Reward hacking, contaminação dos testes, overfitting a benchmarks, mudanças no comportamento do professor, regressões silenciosas, incompatibilidade de arquitetura, dependência de dados sintéticos e seleção de hipóteses favoráveis. O programa deve publicar experimentos que não funcionaram.

> Limite inegociável
>
> O loop de melhoria não altera autonomamente permissões, identidade, políticas, chaves, filtros de segurança, logs, limites de ferramentas ou rotas de aprovação. Mudanças nesses controles são tratadas como alterações de segurança com autorização externa.

24. Aprendizagem contínua e proteção contra esquecimento

A aprendizagem contínua será tratada como problema de pesquisa específico. O sistema deve aprender conhecimento incremental sem destruir capacidades já demonstradas. A literatura identifica interferência e esquecimento catastrófico como desafios centrais [14].

Estratégias

- Replay de exemplos representativos com proteção de privacidade e licença.
- Adaptadores por domínio e roteamento de adapters, evitando atualização global quando não necessária.
- Regularização baseada em importância de parâmetros, quando estimável.
- Memória externa e recuperação, reduzindo a necessidade de alterar pesos.
- Currículos incrementais e replay de tarefas antigas.
- Distilação para checkpoint estudante e comparação de retenção.

Métrica de retenção

```
F_t = mean_j (score_j,baseline − score_j,after_t)
```

F_t mede perda média em um conjunto de tarefas anteriores. Medir também backward transfer, forward transfer, pior regressão individual e mudança de calibração. A métrica média não pode ocultar regressão grave em uma tarefa de alto risco.

Critério de aceite

Toda atualização que modifica pesos ou adaptadores deve superar o baseline em pelo menos uma métrica primária e permanecer dentro de limiares predefinidos em toda a bateria de retenção, segurança, robustez e custo.

25. Estrutura de software e repositório

A primeira implementação deve ser um monorepo modular com contratos internos estáveis, CI, testes e ambientes separados. A separação de diretórios não substitui a separação de permissões e processos.

Estrutura proposta

```
LRG-SOVEREIGN-AGI/
core/ cognitive/ relational_engine/ affect_engine/
reasoning_engine/ memory_engine/ metacognition/
weight_intelligence/ model_orchestrator/ learning/
world_model/ simulation/ evaluation/ security/
governance/ interfaces/ infrastructure/ datasets/
experiments/ documentation/ tests/
```

Responsabilidades

- core: identidade de tarefa, eventos, schemas, política e contratos.
- model_orchestrator: adaptadores de fornecedor, roteamento, budget e traces.
- weight_intelligence: inspeção autorizada, métricas, hooks e W2M.
- relational_engine: entidade, grafo, regra, validade e contradição.
- affect_engine: vetor de estado de trabalho e política de exploração.
- memory_engine: persistência, recuperação, consolidação e retenção.
- learning/evaluation: datasets, branches, treinamento, regressão e aprovação.
- security/governance: IAM, políticas, auditoria, riscos e incidentes.

Padrões de integração

Python para pesquisa e orquestração; Rust para gateway, parser e serviços de alta integridade; C++/CUDA apenas para kernels que justifiquem otimização. Cada módulo expõe interface tipada, health check, readiness, métricas, logs estruturados e versão do schema.

26. Stack tecnológica recomendada

A seleção deve seguir maturidade operacional, compatibilidade de modelos, controle de dependências, desempenho e disponibilidade de profissionais. A tabela é uma arquitetura de referência, não uma obrigação de usar todo o catálogo.

Tecnologia	Uso proposto	Justificativa e ressalvas
Python	Orquestração, pesquisa, avaliação, dados	Ecossistema de IA; controlar dependências e isolar execução de código.
PyTorch	Treino, inferência e hooks	Biblioteca de tensores e autograd; versão fixada e imagens assinadas [18].
JAX	Experimentos numéricos e compilação	Adequado a pesquisa e computação funcional; avaliar equipe e suporte de modelos.
Transformers	Carregamento/modelos compatíveis	Usar com versões e código remoto desabilitado por padrão.
vLLM	Servir modelos locais	Alto throughput e APIs compatíveis; proxy, IAM e rede devem proteger endpoints [19].
Safetensors	Armazenar pesos	Preferir formatos de tensores sem execução de código; verificar hash e origem.
ONNX	Interoperabilidade de inferência	Converter apenas quando operador e precisão mantiverem comportamento aceitável.
PostgreSQL + pgvector	Metadados, transações, vetores	Boa base relacional para protótipo; validar índices e escala [20].
Banco de grafos	Consulta relacional profunda	Escolher após benchmark; evitar duplicação de verdade entre dois bancos.
Rust / C++	Gateway, ingestão, kernels	Usar em fronteiras de segurança e gargalos medidos; não como linguagem universal.
Kubernetes	Orquestração de serviços	Não substitui escalonador HPC nem fabric RDMA; limitar superfície administrativa.
OpenTelemetry	Traces, métricas e logs	Traçar chamadas de modelo e toolchain; redigir dados sensíveis.

NOTA DE ENGENHARIA

Kubernetes, drivers, runtime GPU e bibliotecas de comunicação precisam de matriz de compatibilidade versionada; atualizar tudo em conjunto sem testes pode quebrar o cluster.

27. Interfaces e API de referência

Os contratos do protótipo podem ser implementados com modelos tipados e validação no limite do serviço. O exemplo abaixo ilustra somente a fronteira de requisição; a execução real exige adaptadores, timeout, limites e trilha de auditoria.

Exemplo de objeto de tarefa

```
TaskRequest = {task_id, prompt, domain, risk_level, modalities, deadline, cost_budget,
data_policy, required_evidence}
```

Pseudocódigo de roteamento

```
route(task):
policy = authorize(task, principal)
candidates = catalog.filter(policy, modality, domain)
ranked = evaluate_routes(candidates, task)
plan = minimize_cost(ranked, quality_floor, risk_ceiling)
results = execute_with_trace(plan, timeout, sandbox)
evidence = extract_claims(results)
verified = run_independent_checks(evidence)
return synthesize_or_abstain(verified, policy)
```

Interface de cada motor

Entrada: envelope tipado, contexto mínimo, política, limite de custo e trace_id. Saída: resultado estruturado, métricas, artefatos, evidências, incertezas e status (success, partial, abstain, error). Toda falha técnica é explícita e monitorada; não pode ser convertida em sucesso textual.

Timeout e idempotência

Tarefas de pesquisa recebem idempotency_key; serviços longos usam filas duráveis; retentativas com backoff e limite; cancelamento propagado; deduplicação de resultados. Operações externas com efeito exigem escopo e confirmação humana por política.

28. Exemplo Python: contrato de experimento

Um contrato mínimo de experimento torna resultados comparáveis. A implementação real deverá validar schemas, registrar hashes e gravar os artefatos em armazenamento imutável.

Código de referência

```
from dataclasses import dataclass
from typing import Mapping, Sequence

@dataclass(frozen=True)
class ExperimentSpec:
experiment_id: str
baseline_hash: str
candidate_hash: str
dataset_version: str
seeds: Sequence[int]
primary_metric: str
regression_limits: Mapping[str, float]
safety_policy_version: str

def accept_candidate(baseline, candidate, limits):
# Scores must come from the same frozen evaluation harness.
for metric, max_drop in limits.items():
drop = baseline[metric] - candidate[metric]
if drop > max_drop:
return False, f"regression:{metric}"
if candidate["safety_pass"] is not True:
return False, "safety_gate_failed"
if candidate["primary_gain_ci_lower"] <= 0:
return False, "gain_not_significant"
return True, "eligible_for_human_review"
```

Requisitos para tornar executável

- Validar entradas e saídas com Pydantic ou JSON Schema.
- Guardar valores brutos, intervalo de confiança, versão do avaliador e seed.
- Executar cada candidate em ambiente sem credenciais de produção.
- O retorno “eligible” não implanta; apenas habilita revisão humana.
- Adicionar testes unitários, property-based tests e um teste que confirme rollback.

NOTA DE ENGENHARIA

O código é um contrato ilustrativo: para uso real, registrar hashes criptográficos, assinatura de artefatos, log append-only e identidade do aprovador. Não colocar segredos em dataclasses ou traces.

29. Manual de implementação: ambiente inicial

A primeira entrega deve executar em uma estação de trabalho ou servidor controlado antes de migrar para cluster. A instalação deve ser reproduzível e não depender de chaves em arquivos versionados.

Passos

- Criar ambiente Linux suportado pela matriz de GPU. Fixar driver, CUDA/ROCm, Python e pacote.
- Criar ambiente virtual e instalar dependências a partir de lockfile assinado.
- Configurar secrets manager; nunca salvar tokens de API em Git, notebooks ou logs.
- Configurar banco de metadados, objeto e fila; ativar TLS e backups cifrados.
- Instalar modelo local de pequeno porte com licença verificada e hash validado.
- Conectar um fornecedor por API via gateway com proxy, rate limit e política de dados.
- Executar bateria inicial em dataset público licenciado e conjunto interno não sensível.

Estrutura de configuração

```
config/
environment.yaml
model_catalog.yaml
policies.yaml
evaluation.yaml
retention.yaml
```

Critério de saída

A equipe deve reproduzir uma resposta, seu trace, custo estimado, evidências, avaliação e resultado em ambiente limpo, sem edição manual de artefatos. O mesmo experimento deve produzir resultados dentro da tolerância estatística definida.

30. Manual de implementação: integração dos motores

A integração deve seguir a ordem que reduz risco e cria infraestrutura reutilizável. O motor de memória e o sistema de avaliação são a base para testar os demais componentes.

Sequência recomendada

- Implementar core/eventos e controle de acesso.
- Implementar orquestrador com dois modelos e um baseline determinístico.
- Implementar Evaluation Engine antes de qualquer ciclo de modificação.
- Adicionar memória com proveniência e política de retenção.
- Adicionar RAGI em modo “proposta”, depois habilitar inferência tipada.
- Adicionar Affect Engine como scheduler de exploração, sem influência em políticas de segurança.
- Adicionar WIE em modelos de laboratório e somente leitura.
- Adicionar branches de LoRA e melhoria experimental após os testes anteriores.

Plano de testes de integração

Testar schemas inválidos, timeout de API, divergência entre modelos, banco indisponível, dados maliciosos, repetição de eventos, entidade ambígua, contradição temporal, alteração de hash, tentativa de execução de ferramenta sem autorização e falha durante rollback.

Implantação controlada

A implantação usa canário, feature flags e monitoramento em tempo real. A equipe de produção recebe apenas artefatos aprovados. Um plano de reversão mantém o último snapshot íntegro e o caminho de reprocessamento de eventos.

31. Infraestrutura: dimensionamento por cenário

O programa não deve treinar do zero modelos de fronteira. O dimensionamento prioriza inferência de modelos selecionados, fine-tuning de adaptadores, análise de pesos, avaliação e simulações. Valores são envelopes de arquitetura; a compra exige prova de conceito com modelos e cargas reais.

Premissas

A memória da GPU, bandwidth, quantização, tamanho do batch, janela de contexto e concorrência determinam capacidade real. “Número de GPUs” não basta: medir tokens/s, p95 de latência, taxa de falha e custo por tarefa com modelos reais. Para grandes MoE, usar serving otimizado e shards; evitar réplicas ociosas sem plano de carga.

> Regra de aquisição
> Não comprar cluster de produção antes de medir a utilização dos aceleradores no protótipo e validar a demanda de workloads do órgão.

Parâmetro	Laboratório inicial	Centro experimental	Produção escalável
Aceleradores	8–16 GPUs 80–192 GB classe datacenter	64–128 GPUs classe H100/H200/B200 ou equivalentes	512–1.024+ GPUs em clusters segregados
CPU/RAM	2–4 nós; 512 GB–1 TB/nó	8–16 nós; 1–2 TB/nó	dezenas de nós; RAM distribuída conforme workload
Armazenamento	100–300 TB bruto; 50–150 TB NVMe	2–4 PB objeto; 250–500 TB NVMe	10–30 PB objeto; NVMe distribuído
Rede	100–200 Gb/s; fabric GPU apropriado	200–400 Gb/s RDMA/IB ou equivalente	400–800 Gb/s backbone conforme arquitetura
Potência TI	20–40 kW	150–250 kW	1,0–2,0 MW
Uso principal	Prototipagem e pequenos adaptadores	Avaliação em escala, fine-tuning e multimodelo	Inferência nacional, simulação e modelos locais

32. Infraestrutura: energia, refrigeração e continuidade

A operação deve ser projetada como instalação crítica de TI, com redundância, segurança física, refrigeração compatível, proteção contra incêndio, controle de acesso e continuidade. O PUE real deve ser medido, não presumido.

Estimativa energética

```
Energia anual (kWh) = Potência TI (kW) × 8.760 × PUE × fator médio de carga
```

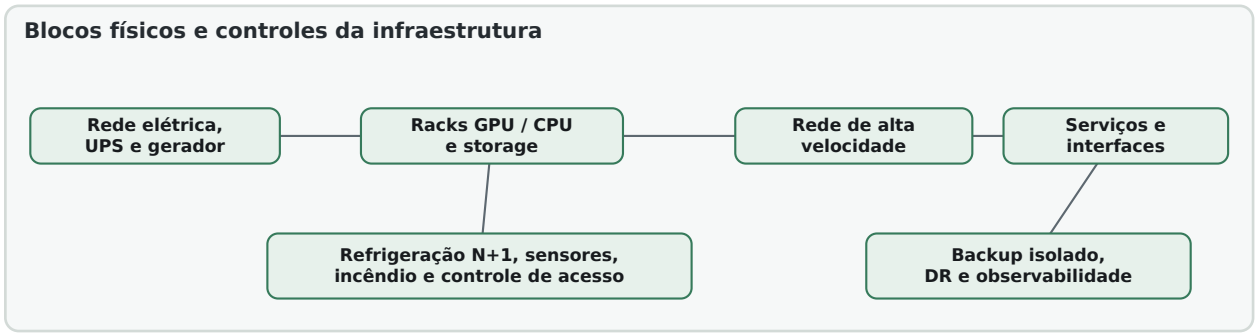
Exemplo de centro de 200 kW TI, PUE 1,35 e fator de carga 0,70: 200 × 8.760 × 1,35 × 0,70 ≈ 1,66 GWh/ano. Com tarifa hipotética de R$ 0,80/kWh, o custo simples seria cerca de R$ 1,33 milhão/ano antes de demanda, impostos, contratos, bandeiras, perdas e tarifas específicas. O preço de R$ 0,80 é somente hipótese de sensibilidade, não cotação.

Requisitos

- UPS e geradores calculados para carga crítica, autonomia e plano de combustível.
- Redundância N+1 para refrigeração e componentes de rede essenciais.
- Telemetria térmica por rack, GPU, circuito de água e ambiente.
- Backup isolado e restauração testada; cópias imutáveis protegidas de ransomware.
- Plano de continuidade com local alternativo e testes de recuperação.

Eficiência

Aplicar agendamento de jobs, quantização, batching, cache seguro, desligamento de recursos ociosos e localização próxima a energia de menor custo e menor intensidade de carbono, respeitando soberania e latência.



A potência, o resfriamento e a redundância devem ser dimensionados a partir do workload medido.

33. Plano financeiro de 60 meses: premissas

Os envelopes financeiros abaixo são estimativas paramétricas para decisão de portfólio, não preços contratáveis. Não há cotação única verificável de GPU, servidor, data center, mão de obra ou API aplicável a toda modalidade de contratação. Cada linha exige pesquisa de preços, proposta técnica, tributos, importação e análise de TCO.

Câmbio de referência

Para comparação, foi usada a PTAX de venda de 03/11/2026: R$ 5,4935 por US$ 1 [21]. O orçamento final deve manter rubricas em moeda original, aplicar hedge ou margem cambial e atualizar a cotação na data da contratação.

Premissas de pessoal

Custos médios carregados por FTE incluem salário ou bolsa, encargos, benefícios, recrutamento, gestão e overhead institucional. A faixa depende de senioridade e forma de contratação. O centro experimental adota equipe média de aproximadamente 100 FTE ao longo do programa; a expansão nacional pode superar 250 FTE.

Custeio de modelos

O consumo de APIs deve ser calculado por logs reais de tokens, modalidade, cache, batch, retries e avaliação em duplicidade. O orçamento prevê limites e contingência; preços publicados por fornecedor mudam e não devem ser extrapolados para cargas ainda não medidas.

Contingência

Adotar contingência de 15–20% conforme incerteza técnica, importação, câmbio, licenciamento, atrasos e inflação. O orçamento de expansão inclui custo de segurança, auditoria, governança e manutenção, não apenas compra de aceleradores.

34. Orçamento comparativo: três cenários

Valores arredondados em reais correntes de planejamento, para cinco anos. As faixas são envelopes a serem refinados por orçamento-base em Fase 0; valores em dólares são apenas conversão pela PTAX citada.

Leitura correta

O cenário conservador valida arquitetura e resultados sem montar capacidade nacional. O intermediário financia um centro experimental com 64–128 GPUs, equipes completas e pilotos. A expansão nacional exige política de localização, operação multiorganização, redundância de instalações e uma capacidade de compute muito superior.

> Recomendação orçamentária
>
> Liberar recursos em parcelas por marcos técnicos. Não aprovar o cenário nacional integral antes da avaliação independente ao final de 24 e 36 meses.

Rubrica (R$ milhões)	Conservador	Intermediário	Expansão nacional
Pessoal e bolsas	45–65	160–220	550–800
Hardware e implantação (CAPEX)	40–70	200–320	1.200–1.800
Compute cloud / APIs / licenças	20–35	80–150	250–450
Energia / instalações / manutenção	12–20	50–90	180–320
Segurança, auditoria e governança	8–15	25–45	90–150
Dados, validação, jurídico e contratação	5–10	15–30	50–100
Contingência e risco	20–35	100–180	450–700
Total indicativo	150–250	630–1.035	2.770–4.320
US$ equivalente aproximado	US$ 27–46 mi	US$ 115–188 mi	US$ 504–786 mi

35. Perfil de desembolso e custo total de propriedade

O TCO deve separar investimento inicial, custo recorrente, custo de pessoal, custo de oportunidade e custo de encerramento. Uma infraestrutura comprada pode sair mais cara que nuvem em períodos de baixa utilização; nuvem pode gerar dependência, custos imprevisíveis e risco de transferência de dados.

Distribuição temporal sugerida

Ano 1: 12–16% do total, concentrado em equipe fundadora, desenho, ambiente seguro e pequeno cluster. Ano 2: 18–22%, integração e capacidade de avaliação. Ano 3: 22–25%, WIE, aprendizagem e infraestrutura expandida. Ano 4: 22–25%, validação e piloto. Ano 5: 18–22%, operação piloto, auditoria, transferência e decisão de escala.

Critérios econômicos

- Custo por tarefa concluída e aceita, não apenas custo por token.
- Custo de auditoria, validação humana e tratamento de incidentes.
- Utilização média e p95 de ocupação dos aceleradores.
- Custo de egress, armazenamento, cópias, energia e resfriamento.
- Custo de atualização e substituição de modelo/provedor.
- Valor de saída: documentação, dados licenciados, software, know-how e capacidade de transferência.

Matriz build / buy / partner

Construir core cognitivo, avaliação, governança e camada de integração. Comprar hardware e serviços commodity com competição. Contratar APIs para acesso a modelos que não podem ser implantados localmente. Firmar parcerias acadêmicas para avaliação e interpretabilidade. Exigir portabilidade e cláusulas de saída em todos os contratos.

36. Equipe e estrutura institucional

A equipe deve crescer por marcos. Um programa de pesquisa não se mede apenas por quadro de pessoal: a qualidade dos líderes, avaliação independente, capacidade operacional e governança têm peso semelhante ao número de engenheiros.

Disciplina	Fase inicial (FTE)	Centro experimental (FTE)	Expansão (FTE)	Responsabilidade principal
Direção científica / arquitetura	2–3	4–6	8–12	Visão, prioridades, hipóteses e integração científica.
ML / modelos / dados	4–6	18–25	45–65	Treino, inferência, adaptadores, dados e teacher models.
WIE / interpretabilidade	2–3	8–12	20–30	Pesos, ativações, SAE, intervenções e análise de mudança.
Matemática / RAGI / causalidade	2–3	8–12	18–28	Formalização, grafos, prova, inferência e modelos causais.
Software / HPC / SRE	5–7	20–25	50–65	Plataforma, serving, clusters, confiabilidade e performance.
Segurança / privacidade	2–3	6–8	15–22	Threat model, IAM, resposta a incidentes e LGPD.
Avaliação / auditoria	2–3	8–10	20–28	Benchmarks, testes cegos, segurança e reprodutibilidade.
Produto público / domínio	1–2	6–8	15–25	Casos de uso, requisitos de órgão e avaliação de impacto.
Gestão / jurídico / compras	2–3	5–8	12–18	Contratação, orçamento, licença, governança e compliance.

NOTA DE ENGENHARIA

FTE são equivalentes em tempo integral e representam quadro médio, não número de contratações individuais. Especialistas externos e universidades podem compor avaliações independentes.

37. Governança e órgãos de decisão

A estrutura institucional deve separar a equipe que desenvolve de quem aprova mudanças relevantes. O programa deve adotar um sistema de gestão de IA, inspirado em ISO/IEC 42001 e gestão de risco de IA NIST, adaptado ao contexto público [22, 23].

Comitês

- Comitê Diretor: orçamento, escopo, contratação e decisão de continuidade.
- Conselho Científico: hipóteses, método, integridade acadêmica e prioridades.
- Comitê de Segurança: threat model, permissões, incidentes e aprovação de alterações críticas.
- Comitê de Dados e Direitos: privacidade, direitos fundamentais, avaliação de impacto e participação pública.
- Painel de Avaliação Independente: benchmark, reprodução, auditoria e decisão sobre marcos científicos.

Gates de autorização

G0: experimentação com dados sintéticos/públicos. G1: uso interno não sensível. G2: dados institucionais com autorização. G3: piloto operacional de baixo risco. G4: aplicação mais ampla após auditoria, análise jurídica e aprovação do órgão. O nível de risco não pode ser reduzido por autoavaliação do próprio modelo.

Responsabilização

Toda versão de sistema deve ter proprietário institucional, responsável técnico, aprovador, documentação de uso pretendido, usos proibidos, limites conhecidos, monitoramento e plano de retirada. O registro de decisão deve sobreviver ao ciclo de vida do fornecedor e da equipe.

38. Segurança arquitetural e threat model

O sistema tem superfícies de ataque: prompts, documentos, ferramentas, APIs, pesos, datasets, bibliotecas, containers, rede, identidades e operadores. A segurança precisa ser aplicada no desenho, não adicionada apenas como filtro de saída.

Ameaças prioritárias

- Prompt injection e conteúdo recuperado que tenta alterar instruções.
- Exfiltração de dados via logs, traces, ferramentas, prompts ou fornecedores.
- Supply chain: checkpoint malicioso, pacote comprometido, imagem adulterada ou dependência vulnerável.
- Data poisoning e contaminação de benchmark.
- Abuso de ferramenta, escalada de privilégio e execução arbitrária.
- Roubo de credenciais, chave ou segredo de API.
- Modificação não autorizada de pesos, políticas, bancos e trilha de auditoria.

Controles

Zero trust; IAM com privilégio mínimo; rede segmentada; serviço proxy; allowlist de ferramentas; sandbox sem secrets; egress controlado; criptografia em repouso e trânsito; KMS/HSM; assinatura e SBOM; varredura de vulnerabilidades; revisão de código; logs imutáveis; backups isolados; rate limits; detecção de anomalia; red-team e pentest.

Aprovação de ferramentas

Cada ferramenta exige manifesto com capacidade, escopo, dados acessados, efeitos colaterais, timeout, limites, identidade técnica, proprietário humano e teste de abuso. A ferramenta não deve aceitar uma ordem gerada em conteúdo recuperado sem autorização do plano de controle.

39. Segurança do ciclo de autoaperfeiçoamento

O pipeline de melhoria precisa ser tratado como uma cadeia de suprimentos de software e modelos. O modelo de produção nunca recebe diretamente o artefato em construção.

Ambientes

- Pesquisa: dados não sensíveis, rede controlada e permissões limitadas.
- Treino: acesso temporário apenas aos datasets autorizados e artefatos necessários.
- Avaliação: conjuntos retidos, sementes independentes e harness assinado.
- Pré-produção: replay de tráfego anonimizado e testes de carga.
- Produção: somente artefatos aprovados, credenciais específicas e rollout reversível.

Assinaturas e hashes

```
artifact_id = SHA256(weights || config || tokenizer || adapter || code_version)
```

Na prática, componentes devem possuir hashes separados e manifesto assinado para permitir atualização parcial e verificação. A assinatura identifica quem aprovou e quando; hash não prova que o artefato é benigno.

Rollback

Rollback automático pode ser disparado por quebra de SLO, aumento de erros, violação de segurança, degradação funcional além do limite ou inconsistência de versão. O procedimento deve restaurar pesos, adaptadores, configuração, índice, política e schema compatíveis.

> Proibição de autonomia irrestrita
>
> A AGI não pode conceder novas permissões a si própria, desativar auditoria, alterar a política de aprovação ou remover controles de segurança.

40. Conformidade jurídica no Brasil: mapa inicial

O enquadramento jurídico é específico por tratamento, órgão, finalidade, categoria de dado e canal de implantação. Esta seção é triagem de engenharia e não substitui parecer da Advocacia Pública ou encarregado de dados.

Normas e status verificados

- Lei nº 13.709/2018 (LGPD): aplica-se ao tratamento de dados pessoais por agentes públicos e privados. Exige finalidade, adequação, necessidade, transparência, segurança, prevenção e responsabilização; avaliar bases legais e papéis do controlador/operador [24, 25].
- Lei nº 12.965/2014 (Marco Civil da Internet): estabelece princípios, garantias, direitos e deveres para uso da internet. A ANPD informou novos decretos regulamentares em 20/05/2026, com competências e obrigações atualizadas para provedores em seus âmbitos [26].
- Lei nº 14.133/2021: referência geral de licitações e contratos administrativos da administração direta, autárquica e fundacional, sujeita a regulamentos e ao ente contratante [27].
- Lei nº 12.527/2011 (LAI) e Lei nº 14.129/2021 (Governo Digital): avaliar transparência, acesso à informação, serviços digitais e restrições de sigilo/dados pessoais.
- PL nº 2.338/2023: na ficha consultada da Câmara em 03/11/2026, aguardava parecer de relator na comissão especial. Não tratar como lei vigente enquanto não promulgado e publicado [28].

Relatório de impacto

ANPD recomenda considerar RIPD em contextos de alto risco, incluindo larga escala, dados sensíveis e determinadas decisões automatizadas que possam afetar direitos ou acesso a serviços [29].

> NOTA DE ENGENHARIA
> Revalidar o status legislativo e atos da ANPD no início de cada fase. A aplicação do Marco Civil e dos decretos de 2026 deve ser detalhada por advogado, especialmente para plataformas e provedores.

41. Conformidade internacional e contratação pública

Referenciais internacionais são instrumentos de gestão e interoperabilidade; não se tornam obrigação jurídica no Brasil por mera citação. A adoção deve ser formalizada por política institucional e adequação ao contrato.

Referenciais

- NIST AI RMF 1.0: funções Govern, Map, Measure e Manage, com gestão contínua de riscos ao longo do ciclo de vida [22].
- NIST AI 600-1: perfil para IA generativa, útil para riscos particulares de geração, conteúdo sintético, integração e interação humana [23].
- ISO/IEC 42001: sistema de gestão de IA; política, objetivos, papéis, avaliação, melhoria contínua e auditoria [30].
- ISO/IEC 23894: orientação de gestão de riscos de IA, a adquirir e aplicar conforme escopo [30].
- Princípios de IA da OCDE, atualizados em maio de 2024: inovação confiável, direitos humanos, transparência, robustez e responsabilização [31].
- Regulamento (UE) 2024/1689 (AI Act): norma da União Europeia com aplicação faseada. Na consolidação consultada, a maior parte das regras e a fiscalização passam a aplicar-se em 02/08/2026; certas obrigações para sistemas de alto risco têm datas posteriores, incluindo 02/12/2027 para sistemas autónomos abrangidos pelo Anexo III e 02/08/2028 para sistemas incorporados em produtos regulados. Não é automaticamente vinculante no Brasil: avaliar o âmbito territorial e material, o mercado-alvo, os papéis de fornecedor/implantador e a cadeia contratual antes de concluir aplicabilidade [41].

Cláusulas contratuais recomendadas

- Propriedade e licença de pesos, código, datasets, outputs e documentação.
- Proibição de treino do fornecedor com dados do órgão sem autorização expressa e base legal.
- Região de processamento, suboperadores, transferências internacionais e retenção.
- Aviso de incidentes, direito de auditoria e evidências de segurança.
- Portabilidade, exportação de logs, plano de saída e assistência de migração.
- Mudança de modelo, preços, depreciação, SLA e janela de manutenção.

> Decisão jurídica
>
> Para cada caso de uso, registrar finalidade pública, base legal, classificação de risco, RIPD quando aplicável, transparência, canal de contestação e responsável pelo ato.

42. Aplicações governamentais prioritárias e limites

A arquitetura pode apoiar pesquisa e planejamento, mas aplicações de alto impacto exigem revisão jurídica e institucional específica. Começar por atividades de baixo risco e benefício mensurável.

Domínio	Aplicação inicial segura	Limite operacional
Pesquisa científica	Busca bibliográfica, síntese, cálculo reproduzível, revisão de código	Pesquisador valida fontes, inferências e autoria.
Energia	Previsão de demanda, cenários, manutenção e otimização	Operador humano mantém autoridade sobre despacho crítico.
Infraestrutura	Inspeção documental, priorização de manutenção, simulação	Engenheiro habilitado aprova decisões estruturais.
Meio ambiente	Modelos de risco, sensoriamento e cenários	Não substituir fiscalização, laudos ou medição in situ.
Logística	Roteamento, alocação, simulação de filas	Validar dados e efeitos distributivos.
Administração	Classificação documental e apoio a processos	Não automatizar decisão final sobre direitos.
Economia	Cenários e análise de sensibilidade	Mostrar premissas; não apresentar projeção como fato.
Emergências	Síntese de informação e planejamento de recursos	Comando humano, protocolos oficiais e redundância.
Saúde, justiça, policiamento e benefícios	Apenas pesquisa não decisória e sob controles reforçados	Sem decisão autônoma sobre pessoas, liberdade ou direitos.

NOTA DE ENGENHARIA

Nenhuma aplicação deve entrar em produção apenas porque o modelo teve boa pontuação de benchmark; exige teste no contexto real, avaliação de impacto, segurança e processo de contestação.

43. Programa experimental: desenho comum

O protocolo científico comum deve ser registrado antes da execução. Cada experimento define hipótese, tarefa, grupo de controle, métrica primária, amostra, repetição, sementes, critérios de exclusão e análise estatística.

Padrões de validade

- Separar treino, desenvolvimento e teste retido. Guardar teste cego fora do alcance de professores e desenvolvedores.
- Fixar versão de modelo, prompt, ferramenta, dados, código, seed e avaliador.
- Relatar média, desvio, IC 95%, tamanho de efeito e teste de hipótese apropriado.
- Controlar multiplicidade quando testar várias intervenções ou hipóteses.
- Usar baseline forte e ablações para cada motor.
- Replicar por pelo menos 5 seeds para protótipos e ampliar amostras em avaliação confirmatória.

Métricas transversais

Acurácia, precisão/recall, calibração, consistência, robustez, generalização, taxa de abstenção, custo por tarefa aceita, latência p50/p95, consumo energético, falhas de segurança, cobertura de proveniência e regressão de tarefas anteriores.

Benchmarks

Usar GPQA, SWE-bench, MMLU-Pro/MMMU-Pro, LiveCodeBench e tarefas científicas específicas como referência, além de conjuntos próprios retidos. Nenhum ranking isolado representa inteligência geral. Os resultados do fornecedor devem ser reproduzidos em harness independente, com prompt, ferramentas, modelo e data publicados [32].

44. Experimentos A–D

Os primeiros quatro experimentos testam a premissa de integração e os motores fundamentais. Os critérios abaixo são propostas de desenho; limiares exatos precisam ser registrados na Fase 0.

ID / hipótese	Variáveis e controles	Métricas / critérios de rejeição
A. Multimodelo: combinação de especialistas melhora qualidade/custo versus melhor modelo único.	Indep.: número de modelos, roteador, domínio. Controle: melhor modelo individual e ensemble aleatório. 5–10 seeds/partições.	Qualidade, custo por tarefa aceita, p95 e calibração. Rejeitar se aumento de qualidade não superar IC ou custo exceder teto.
B. WIE: análises internas ajudam a localizar mudanças associadas a comportamento.	Indep.: análise estatística, SAE, patching, snapshot. Controle: observação black-box e camada aleatória. Pesos autorizados.	Poder preditivo em teste retido, replicação, causalidade de intervenção. Rejeitar se feature/efeito não se replicar.
C. RAGI: relações tipadas melhoram recuperação e rastreabilidade.	Indep.: grafo, regras, evidência, contexto temporal. Controle: RAG vetorial puro e KG sem proveniência.	Precision/recall, contradição, calibração, grounded answer. Rejeitar se falsos positivos aumentarem sem ganho útil.
D. Affect: estado operacional melhora política de exploração e revisão.	Indep.: motor afetivo ligado/desligado, diferentes políticas. Controle: scheduler fixo. Sem dados emocionais pessoais.	Retrabalho, erros, custo e cobertura de validação. Rejeitar se ganho não for replicável ou induzir atalho inseguro.

NOTA DE ENGENHARIA

Todos os experimentos devem ser submetidos à revisão de ética, privacidade e segurança conforme dados, participantes e ambiente envolvidos.

45. Experimentos E–H

A segunda etapa investiga aprendizagem contínua, alterações paramétricas e o ciclo de autoaperfeiçoamento. Mudanças em pesos devem permanecer isoladas e usar somente checkpoints para os quais exista autorização e licença válida.

Independência

O time que propõe a melhoria não deve ser o único time a pontuá-la. O painel independente recebe a versão candidata, conjunto de testes e metodologia, e pode executar avaliações não anunciadas antes de aprovar.

> Critério de ciência
>
> Uma hipótese que falha corretamente reduz incerteza e é um resultado válido. A equipe deve publicar falhas, limitações e mudanças de protocolo.

ID / hipótese	Variáveis e controles	Métricas / critérios de rejeição
E. Continual learning: adaptação incremental retém competências.	Treino por etapa, replay, adapters, regularização. Controle: fine-tuning simples e congelamento total.	Retenção, forward/backward transfer e regressão por tarefa. Rejeitar se queda crítica exceder δ.
F. Alterações paramétricas: updates selecionados podem corrigir falhas específicas.	LoRA/edits, camada-alvo, taxa de update. Controle: sem update e update placebo.	Generalização de edição, side effects, factuality e retenção. Rejeitar efeito colateral fora do teto.
G. Generalização: melhorias transferem para tarefas inéditas.	Domínios e distribuições retidas, tarefas composicionais e adversariais. Controle: treino e validação padrão.	Gap in/out-of-distribution, calibração, robustez e transferência. Rejeitar se apenas benchmark público melhorar.
H. Autoaperfeiçoamento: ciclo controlado seleciona melhorias com ganho líquido.	Ciclo habilitado/desabilitado, budget, métricas e gate humano. Controle: especialista humano e baseline estável.	Ganho líquido, regressões, tentativas rejeitadas, custo, segurança e rollback. Rejeitar se seleção gerar reward hacking ou contaminação.

46. Fluxo de avaliação e indicadores-chave

O Evaluation Engine é a fonte de verdade para a evolução do sistema. Benchmarks, tarefas institucionais e testes de segurança devem compartilhar registro de versão e relatório reproduzível.

Pipeline

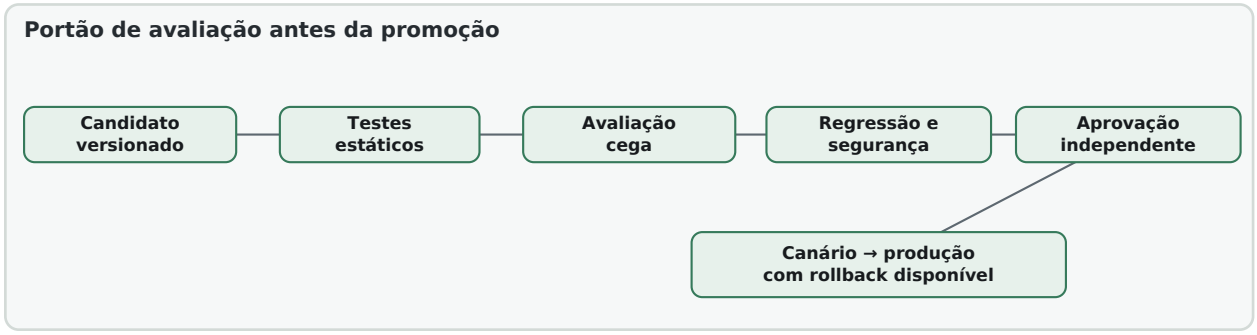
```
candidate → static checks → unit/integration → safety eval → blind task eval → regression
→ human review → canary → production
```

KPIs do programa

- Qualidade: taxa de tarefa concluída corretamente e aceita por avaliador cego.
- Confiabilidade: calibração, taxa de alucinação factual em tarefas com fonte e taxa de abstenção correta.
- Aprendizagem: retenção, transferência e ganho em tarefas novas.
- Interpretabilidade: proporção de análises replicáveis e intervenções causais válidas.
- RAGI: precisão de relações, cobertura de proveniência e contradições não resolvidas.
- Operação: custo por tarefa, latência p95, disponibilidade, falhas e consumo por workload.
- Segurança: incidentes, vulnerabilidades abertas, acessos indevidos e tempo de resposta.

Painel de decisão

Exibir tendência e intervalo, baseline e valor absoluto; separar dados de desenvolvimento e teste; bloquear agregação de métricas que oculte regressões críticas. Indicadores devem ter dono, fonte e periodicidade.



Falha em qualquer gate interrompe a promoção e mantém a versão estável como padrão.

47. Cronograma de 60 meses: mapa de fases

O cronograma é incremental e sujeito a gates. As fases se sobrepõem apenas quando a infraestrutura e a governança já estão disponíveis. Nenhuma fase promete alcançar AGI.

Fase	Mês	Entregável verificável	Gate de continuidade
0. Planejamento e pesquisa	0–6	Arquitetura, análise jurídica, threat model, benchmark inicial e budget refinado.	Comitês implantados; dados, licenças e baseline definidos.
1. Protótipo modular	4–12	Core, schemas, gateway, banco, telemetria e primeiro modelo local/API.	Reprodução end-to-end e rollback testado.
2. Integração multimodelo	9–18	Roteador, catálogo, rastreabilidade e avaliação comparativa.	Ganho ou redução de custo em tarefas-alvo.
3. Motores cognitivos	12–24	RAGI, memória, Affect, raciocínio e metacognição.	Ablation mostra contribuição mensurável.
4. WIE e edição experimental	18–30	Inventário de pesos, W2M, ativações, SAE e branches.	Análises reproduzíveis e autorização de pesos.
5. Aprendizagem contínua	24–42	Adaptadores, replay, retenção e ciclo candidato.	Sem regressões críticas; ganho independente.
6. Validação independente	36–48	Relatório externo e reprodução dos resultados.	Painel aprova limites e maturidade.
7. Piloto governamental	42–54	Piloto delimitado, baixo risco, human-in-the-loop.	Avaliação de impacto e operação segura.
8. Expansão institucional	54–60	Plano de transferência, operação e decisão de escala.	Auditoria, TCO, governança e autorização.

48. Gestão de riscos do programa

A governança de riscos deve ser contínua, multidisciplinar e vinculada aos gates. Probabilidade e impacto serão reavaliados com evidência; a lista inicial não substitui análise específica do órgão.

Risco	Sinal de alerta	Mitigação / dono
Ausência de ganho geral	ganho apenas em benchmark selecionado	testes retidos, avaliação independente / Direção científica
Dependência de fornecedor	API indisponível ou alteração de termos	multi-provedor, modelo local, cláusulas de saída / Arquitetura
Vazamento de dados	dados sensíveis em traces ou prompts	minimização, DLP, proxy e auditoria / Segurança e DPO
Interpretação excessiva de pesos	claims sem intervenção ou replicação	protocolo causal, revisão cega / WIE
Esquecimento catastrófico	regressões após treino incremental	replay, adapters, gates / Aprendizagem
Propagação de inferência falsa	crescimento de relações sem fonte	proveniência obrigatória, estado candidate / RAGI
Reward hacking	métrica melhora e comportamento piora	métrica múltipla, red-team, holdout / Avaliação
Custo/energia fora do plano	GPU ociosa ou API supera budget	metering, TCO, gates de compra / Finanças/HPC
Mudança regulatória	novo decreto, lei ou entendimento ANPD	monitoramento jurídico e revisão de DPIA / Jurídico
Uso indevido em decisão de alto impacto	saída automatizada influencia direito sem revisão	proibição técnica, escalonamento humano / Comitê de Direitos

49. Arquitetura de implantação governamental

A implantação deve separar a zona soberana de dados e sistemas oficiais da zona experimental e das APIs externas. A conexão com provedores deve ocorrer por gateway com filtragem, tokenização e contrato de dados.

Domínios de implantação

- Zona de dados governamentais: sistemas oficiais, documentos e bases com classificação e acesso.
- Zona ACIA core: identidade, orquestração, contratos e política.
- Zona de modelos locais: inferência em pesos aprovados e WIE.
- Zona de experimentação: datasets retidos, treinamento, branches e simulação.
- Zona externa: APIs de fornecedores por gateway e contratos específicos.
- Zona de auditoria: logs append-only, trilhas de aprovação, indicadores e exportação.

Fluxo de dados

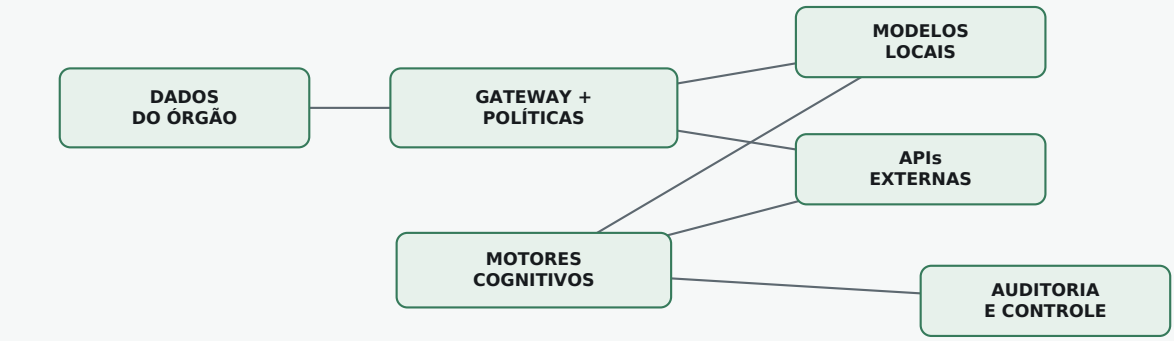
Dados classificados não cruzam zonas sem decisão explícita de política. O gateway verifica finalidade, base legal, localização, minimização e termos de fornecedor. Se um modelo externo não atende requisito de soberania ou retenção, a tarefa fica local ou é bloqueada.

Interoperabilidade

Interfaces padronizadas reduzem acoplamento, mas a portabilidade deve ser testada: schemas, embeddings, ferramentas, adaptadores e formatos não são automaticamente intercambiáveis. Planejar migração por fornecedor a cada ciclo orçamentário.

> Posicionamento institucional
>
> A plataforma recomenda, organiza evidências, simula cenários e documenta incerteza. O agente público competente decide, registra fundamentos e responde pelos atos administrativos.

Zonas e caminhos da plataforma governamental



50. Plano de validação operacional do piloto

O piloto deve escolher um problema administrativo ou científico de baixo risco, com dados autorizados, métricas claras e benefício esperado quantificado. Não iniciar por decisão de benefício, polícia, saúde, justiça ou liberdade.

Critérios para seleção do caso

- Problema com processo atual mensurável e baseline documentado.
- Dados públicos ou institucionais autorizados, minimizados e de qualidade conhecida.
- Resultado revisável por profissional responsável e facilmente contestável.
- Dano potencial limitado e mecanismo de parada.
- Benefício mensurável em tempo, qualidade, cobertura ou custo.

Plano de avaliação

Executar fase sombra sem influência em decisão; avaliar saídas retrospectivas; realizar revisão humana cega; comparar com procedimento atual; medir erro, tempo, custo, cobertura e desigualdade de desempenho; realizar teste de abuso e segurança; disponibilizar canal de feedback.

Condições para ampliar

Somente ampliar quando qualidade for superior ou custo menor sem queda de segurança, a equipe aceitar o workload, auditoria confirmar proveniência, jurídico aprovar o tratamento, o responsável público aceitar a limitação e o mecanismo de contestação estiver funcionando.

51. Resultados esperados e limites científicos

O programa pretende demonstrar capacidades compostas por evidências, não atribuir rótulo de AGI a um sistema apenas por seu tamanho, fluência ou desempenho em tarefas populares.

Capacidades a medir

- Integração de modelos heterogêneos e seleção de especialistas.
- Análise estrutural de parâmetros e ativações autorizados.
- Representação relacional com proveniência e contradição explícita.
- Memória persistente, recuperação e atualização controlada.
- Detecção de incerteza, inconsistência e necessidade de intervenção.
- Aprendizagem incremental com retenção e generalização.
- Geração de candidatos que produzem melhoria medida e reversível.

Não equivalências

Escalar modelos não prova inteligência geral; modificar pesos não implica criar conhecimento novo; simular emoções não demonstra experiência subjetiva; armazenar relações não prova causalidade; resolver benchmarks não demonstra capacidade universal; e obter consenso entre modelos semelhantes não equivale a validação independente.

Critério de maturidade

Uma alegação de capacidade geral exige conjunto amplo, inédito e adversarial, transferência entre modalidades e domínios, continuidade ao longo do tempo, robustez, autonomia limitada sob supervisão e análise independente de riscos. O programa pode concluir que a arquitetura é útil sem concluir que alcançou AGI.

52. Recomendações executivas e decisão solicitada

A recomendação é iniciar como laboratório de pesquisa aplicada com metas de 6, 12, 24 e 36 meses, sem comprometer desde já o orçamento de expansão nacional.

Primeiros 180 dias

- Nomear direção científica, responsável de segurança, encarregado de dados e painel independente.
- Concluir arquitetura executável, threat model, mapa de dados e política de fornecedores.
- Selecionar 2–3 modelos API e 1–2 modelos de pesos licenciados para benchmark.
- Implementar core, orquestrador mínimo, trace, avaliação, banco de evidências e rollback.
- Criar o primeiro dataset retido e protocolo estatístico.
- Demonstrar A/B: modelo individual vs. sistema multimodelo, com custo e qualidade.

Decisão solicitada

Aprovar a Fase 0 e reservar orçamento conservador de planejamento. Autorizar contratação de pesquisa de preços e parceiros científicos; não autorizar processamento de dados pessoais em APIs externas sem aprovação própria.

> Gate de 6 meses
>
> Prosseguir apenas se houver governança implantada, baseline reproduzível, protótipo integrado, política de dados e evidência de valor em pelo menos um caso de uso de baixo risco.

Conclusão

A ACIA transforma uma visão ambiciosa em uma agenda de engenharia que pode ser auditada e corrigida. A arquitetura mantém os motores relacionais, afetivos, de memória, raciocínio e metacognição; incorpora análise interna de pesos quando legal e tecnicamente possível; e faz do autoaperfeiçoamento um processo experimental com gates externos.

O caminho tecnicamente defensável começa com integração e avaliação. A partir daí, cada motor deve provar sua contribuição por ablação, replicação e comparação. A análise de pesos deve ser tratada como ciência experimental. Relações devem preservar tipagem e proveniência. Estados afetivos devem permanecer variáveis instrumentais. Aprendizagem contínua deve demonstrar retenção. O ciclo de melhoria deve permanecer isolado e reversível.

O valor público do programa não está em prometer uma AGI pronta em cinco anos. Está em criar capacidade nacional de pesquisar, testar, governar e operar sistemas cognitivos avançados com independência tecnológica progressiva, transparência e responsabilidade.

> Declaração final do projeto
>
> LRG SOVEREIGN AGI — Uma arquitetura cognitiva artificial integrada, multimodelo, relacional, adaptativa e experimentalmente autoaperfeiçoável. Desenvolvida para ampliar a capacidade humana de descobrir, compreender, planejar e resolver problemas complexos. LUNZ RESEARCH GROUP — LRG.

Referências bibliográficas e fontes oficiais (1/3)

As referências abaixo sustentam decisões técnicas, status regulatórios e inventário inicial de fornecedores. Catálogos, identificadores, preços e licenças são dinâmicos e devem ser recapturados antes de cada contratação.

[1] OpenAI. Models - OpenAI API documentation. Catálogo consultado em 03 nov. 2026.
https://developers.openai.com/api/docs/models

[2] Anthropic. Models overview - Claude Platform Docs. Catálogo e IDs de API consultados em 03 nov. 2026.
https://platform.claude.com/docs/en/models/overview

[3] Google DeepMind. Gemini model cards; Gemini 4 Argon (publicado em 13 out. 2026) e família Gemini 3.8. Consultado em 03 nov. 2026.
https://deepmind.google/models/model-cards/

[4] Meta. Meta Llama model organization / model catalogue. Consultado em 03 nov. 2026.
https://huggingface.co/meta-llama/models

[5] xAI. Grok Models & Pricing - API documentation. Consultado em 03 nov. 2026.
https://docs.x.ai/developers/models

[6] DeepSeek. API changelog; DeepSeek-V4.1-Flash-1017 update. 17 out. 2026.
https://api-docs.deepseek.com/updates/

[7] Alibaba Cloud. Model Studio - Recommended models and model list. Consultado em 03 nov. 2026.
https://help.aliyun.com/en/model-studio/models

[8] Mistral AI. Models overview; Mistral Large 4 e Mistral Small 4, licenças e disponibilidade. Consultado em 03 nov. 2026.
https://docs.mistral.ai/models

[9] Moonshot AI. Kimi K3 model repository and model card; weights and model license. Consultado em 03 nov. 2026.
https://huggingface.co/moonshotai/Kimi-K3/tree/main

[10] Z.ai. GLM 5.3 model availability / API configuration documentation. Consultado em 03 nov. 2026.
https://zcode.z.ai/en/docs/configuration

[11] MiniMax. MiniMax API Platform / M3.1 Flash availability. Consultado em 03 nov. 2026.
https://platform.minimax.io/subscribe/coding-plan

[12] NVIDIA. Nemotron AI Models - official model family, weights and technical reports. Consultado em 03 nov. 2026.
https://developer.nvidia.com/topics/ai/Nemotron

[13] Ouyang, L. et al. Training language models to follow instructions with human feedback. NeurIPS 2022.
https://papers.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html

[14] Parisi, G. I. et al. Continual lifelong learning with neural networks: A review. Neural Networks 113, 54–71, 2019. DOI: 10.1016/j.neunet.2019.01.012.
https://doi.org/10.1016/J.NEUNET.2019.01.012

Referências bibliográficas e fontes oficiais (2/3)

Documentação de infraestrutura, segurança, normas e legislação consultada para a data de corte do plano.

[15] Anthropic. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. 05 out. 2023.
https://www.anthropic.com/news/towards-monosemanticity-decomposing-language-models-with-dictionary-learning

[16] Anthropic. Mapping the mind of a large language model. 21 maio 2024.
https://www.anthropic.com/research/mapping-mind-language-model

[17] Ha, D.; Schmidhuber, J. World Models. 2018.
https://arxiv.org/abs/1803.10122

[18] PyTorch. PyTorch documentation - tensor library for deep learning. Consultado em 03 nov. 2026.
https://docs.pytorch.org/docs/main/

[19] vLLM. Online Serving - OpenAI-compatible server documentation. Consultado em 03 nov. 2026.
https://docs.vllm.ai/en/stable/serving/openai_compatible_server/

[20] pgvector. Open-source vector similarity search for Postgres.
https://github.com/pgvector/pgvector

[21] Banco Central do Brasil. Sistema PTAX, dólar dos EUA, cotação de venda de 03/11/2026: R$ 5,4935.
https://ptax.bcb.gov.br/ptax_internet/consultarUltimaCotacaoDolar.do

[22] NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023.
https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf

[23] NIST. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, 2024.
https://doi.org/10.6028/NIST.AI.600-1

[24] Brasil. Lei nº 13.709, de 14 de agosto de 2018 (LGPD). Portal do Governo Federal.
https://www.gov.br/pt-br/lgpd/lei-no-13-709-18-lei-geral-de-protecao-de-dados-lgpd

[25] ANPD. Guia orientativo: Tratamento de dados pessoais pelo Poder Público. Publicado em 22 nov. 2024; atualizado em 23 jan. 2025.
https://www.gov.br/anpd/pt-br/centrais-de-conteudo/materiais-educativos-e-publicacoes/guia_orientativo_tratamento_de_dados_pessoais_pelo_poder_publico

[26] ANPD. Marco Civil da Internet: atualizações regulatórias comunicadas em 2026, incluindo Decretos nº 12.975 e 12.976, de 20 maio 2026.
https://www.gov.br/anpd/pt-br/assuntos/marco-civil-da-internet

[27] Brasil. Lei nº 14.133, de 1º de abril de 2021 (Lei de Licitações e Contratos Administrativos).
https://www.gov.br/compras/pt-br/nllc

[28] Câmara dos Deputados. PL 2338/2023 - ficha de tramitação; status consultado em 03 nov. 2026.
https://www.camara.leg.br/proposicoesWeb/fichadetramitacao?idProposicao=2487262

Referências bibliográficas e fontes oficiais (3/3)

Trabalhos de pesquisa, fontes regulatórias e documentação dos provedores usados para compor este plano. A referência a um método não demonstra que sua aplicação à ACIA já tenha sido validada.

[29] ANPD. Relatório de Impacto à Proteção de Dados Pessoais (RIPD).
https://www.gov.br/anpd/pt-br/canais_atendimento/agente-de-tratamento/relatorio-de-impacto-a-protecao-de-dados-pessoais-ripd

[30] ISO. ISO/IEC 42001:2023 - Artificial intelligence management systems; inclui referência à ISO/IEC 23894.
https://www.iso.org/standard/42001

[31] OECD. OECD AI Principles; atualização de maio de 2024.
https://oecd.ai/en/ai-principles

[32] Evals Report. Benchmark directory by task with links to original evaluation repositories. Consultado em 03 nov. 2026; usar como índice, não como ranking oficial.
https://evals.report/Benchmarks

[33] Meng, K. et al. Locating and Editing Factual Associations in GPT. arXiv:2202.05262, 2022.
https://arxiv.org/abs/2202.05262

[34] Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
https://arxiv.org/abs/2005.11401

[35] Hogan, A. et al. Knowledge Graphs. ACM Computing Surveys, 2021.
https://doi.org/10.1145/3447772

[36] Vaswani, A. et al. Attention Is All You Need. NeurIPS 2017.
https://arxiv.org/abs/1706.03762

[37] ISO. ISO/IEC 23894:2023 - Information technology - Artificial intelligence - Guidance on risk management.
https://www.iso.org/standard/77304.html

[38] Brasil. Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet), referenciada em páginas oficiais do Governo Federal e ANPD.
https://www.gov.br/anpd/pt-br/assuntos/marco-civil-da-internet

[39] DeepSeek. Models & Pricing; valores de DeepSeek-V4.1-Flash-1017 e ciclo peak/off-peak. Consultado em 03 nov. 2026.
https://api-docs.deepseek.com/quick_start/pricing/

[40] Amazon Web Services. Amazon Bedrock model card — Moonshot AI Kimi K3; preço de inferência Global Standard e identificador. Consultado em 03 nov. 2026.
https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-moonshot-ai-kimi-k3.html

[41] União Europeia. Regulamento (UE) 2024/1689 (AI Act), texto consolidado consultado em 03 nov. 2026; cronograma de aplicação faseada.
https://eur-lex.europa.eu/eli/reg/2024/1689/2026-07-27

Apêndice A. Glossário de termos e maturidade

Este glossário padroniza o significado operacional das expressões usadas no programa.

Termo	Definição no programa
AGI	Objetivo de pesquisa amplo; não atribuído ao sistema sem evidência de capacidade geral robusta.
ACIA	Adaptive Cognitive Intelligence Architecture, arquitetura proposta pelo LRG.
WIE	Weight Intelligence Engine: ferramenta de análise de parâmetros/ativações de modelos acessíveis.
W2M	Weight-to-Markdown: documentação interpretável de análises, não tradução literal dos pesos.
RAGI	Motor relacional experimental do LRG, baseado em relações tipadas, proveniência e inferência.
Affect Engine	Variáveis operacionais de exploração, incerteza, urgência, fricção e progresso.
Feature	Direção/subespaço de ativação aprendido ou identificado; sem significado intrínseco garantido.
Activation patching	Intervenção controlada em ativações para testar contribuição causal em uma métrica.
Continual learning	Aprendizado em sequência com controle de interferência e esquecimento.
Rollback	Restauração de snapshot anterior validado, incluindo dados/configuração relacionados.
Open weights	Pesos disponíveis, mas licença e direitos de uso continuam a exigir avaliação.
Soberania	Controle institucional de dados, operação, chaves, contratos, auditoria e continuidade.

Apêndice B. Formalização consolidada

As equações são especificações propostas de engenharia. Algumas adaptam técnicas estabelecidas; outras são funções candidatas do LRG a serem avaliadas. Nenhuma equação, por si só, prova capacidade cognitiva.

Orquestração e combinação

$$\alpha_i(x)=\mathrm{softmax}_i(s_i(x)/\tau);\ \mathrm{plan}^*=\arg\min_{\mathrm{plan}} \mathrm{Cost}(\mathrm{plan})\ \text{subject to}\ \mathrm{Quality}(\mathrm{plan})\geq q_0$$

Memória e relações

$$M_{t+1}=\mathrm{Consolidate}(M_t,\mathrm{events},\mathrm{utility},\mathrm{confidence},\mathrm{retention})$$

$$R_1(x,y)\ \wedge\ R_2(y,z)\ \wedge\ \mathrm{Compatible}(R_1,R_2)\ \Rightarrow\ R_3(x,z)$$

Estados computacionais

$$E_{t+1}=\mathrm{clip}(\Lambda E_t+(I-\Lambda)g(O_t,G_t)-d\odot E_t,0,1)$$

Análise neural

$$D\theta=||\theta_2-\theta_1||_2/(||\theta_1||_2+\varepsilon);\ W=U\Sigma V^{\top}$$

$$\mathrm{CKA}(H_1,H_2)=||H_1^{\top}H_2||^2_F/(||H_1^{\top}H_1||_F\ ||H_2^{\top}H_2||_F)$$

Autoaperfeiçoamento e mundo

$$J(\theta)=w_AA+w_GG+w_RR+w_CC-w_KK-w_DD$$

$$s_{t+1}=T(s_t,a_t,\varepsilon_t)$$

Calibração

$$\mathrm{Brier}=(1/N)\Sigma_i(p_i-y_i)^2$$

As equações são válidas apenas sob as hipóteses definidas nas seções correspondentes; por exemplo, CKA requer conjuntos de amostras alinhados, e Dθ requer parâmetros comparáveis. Documentar domínio, tolerância numérica e fonte de cada equação em implementação.

Apêndice C. Checklist de revisão técnica e pré-produção

Lista de verificação de encerramento antes de liberar qualquer módulo ou piloto.

Arquitetura e dados

- Schemas, contratos e dependências aprovados e versionados.
- Origem, licença, finalidade, classificação e retenção dos dados documentadas.
- Identidade de modelos, pesos, adapters, prompts e índices registrada.

Segurança

- IAM de privilégio mínimo, secrets manager, TLS, criptografia e rede segregada.
- Sandbox e egress controlado; ferramenta testada contra abuso.
- SBOM, assinatura, varredura de dependências e hashes verificados.
- Logs append-only, monitoramento, backup, restauração e incidente ensaiados.

Avaliação e governança

- Baseline, conjunto retido, seeds e avaliador independente definidos.
- Testes de regressão, segurança, robustez e calibração aprovados.
- RIPD/avaliação de impacto e parecer jurídico feitos quando aplicáveis.
- Responsável público, canal de contestação e processo de revisão humana definidos.
- Canário, limites de operação, rollback e critérios de parada testados.

> Condição final
>
> Nenhuma aprovação técnica substitui a autorização jurídica, administrativa ou de segurança exigida para o contexto real de implantação.

Apêndice D. Questões de pesquisa que permanecem abertas

As perguntas abaixo definem uma agenda científica original para o LRG, com critérios de falseabilidade. Devem ser convertidas em hipóteses específicas antes de experimentos.

- Quais medidas de estrutura de pesos melhor predizem mudanças comportamentais em modelos com arquitetura alinhada?
- Quais features aprendidas por SAE mantêm estabilidade entre seeds, checkpoints e mudanças de domínio?
- Em que condições a composição de relações RAGI produz inferências úteis sem elevar falsos positivos?
- Estados computacionais de curiosidade e incerteza reduzem custo de busca ou apenas redistribuem tentativas?
- Que combinação de replay, adaptadores, regularização e memória externa maximiza retenção e transferência?
- É possível prever regressões de capacidade por estatísticas internas antes de avaliações comportamentais?
- O ciclo autoaperfeiçoável encontra melhorias que avaliações independentes confirmam em tarefas inéditas?
- Quais limites de autonomia, monitoramento e autoridade humana minimizam risco operacional em ambiente público?

A resposta a essas perguntas pode justificar, modificar ou encerrar componentes da ACIA. A arquitetura é uma hipótese de projeto e deve permanecer revisável à luz dos resultados.